



FORMATION INFORMATIQUE - EQUIPE

Git - En equipe

Flux, revue, rebase, CI et releases :
collaborer sans se marcher dessus.

DanielCraft - Livre debutant

Sommaire

Clique un chapitre ou un sous-chapitre pour y aller.

1. Chapitre 1 - Rappel rapide et carte du livre

- Ce que ce n'est pas
- Ce que tu vas savoir faire
- Comment lire ce livre
- Petite histoire
- Erreur classique
- En vrai
- A toi
- Petite scene DanielCraft

2. Chapitre 2 - Flux d'equipe : qui pousse ou, quand tirer

- Une journee type a trois
- Quand tirer (pull)
- Petite histoire
- Erreur classique
- En vrai
- A toi

3. Chapitre 3 - Strategie de branches simple (main + feature)

- Noms et duree de vie
- Ce que ce n'est pas (encore)
- Petite histoire
- Erreur classique
- En vrai
- A toi

4. Chapitre 4 - Rebase vs merge : idee, quand, risques

- Un cas concret : Lea met a jour sa feature
- Quand le merge est plus simple (et plus sur)
- Petite histoire
- Erreur classique
- En vrai
- A toi

5. Chapitre 5 - Historique propre : messages, petits commits, squash

- L'idee du squash
- Petite histoire
- Erreur classique
- En vrai
- A toi

6. Chapitre 6 - Revue de code humaine

- Le ton qui aide
- But
- Changements

- [Comment tester](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

7. [Chapitre 7 - Branches protegees : pas de push direct sur main](#)

- [Ce que ca change au quotidien](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)
- [Zoom : filet visible, egos calmes](#)

8. [Chapitre 8 - CI legere : des tests automatiques sur la PR](#)

- [L'idee sans jargon](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)
- [Zoom : vert lisible, rouge utile](#)

9. [Chapitre 9 - Tags et releases](#)

- [Release sur GitHub](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)
- [Zoom : nommer pour retrouver](#)

10. [Chapitre 10 - Cherry-pick : reprendre un commit ailleurs](#)

- [Conflits et limites](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)
- [Zoom : urgence sans chaos](#)

11. [Chapitre 11 - Bisect : trouver le commit qui a casse](#)

- [Le deroulement](#)
- [Automatiser un peu](#)
- [Bien choisir good et bad](#)
- [Ce que bisect n'est pas](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

12. [Chapitre 12 - Mini-projet : simulation d'equipe](#)

- Preparation
- Role Lea : feature tarifs
- Role Max : fix email
- Role Sam : filets et release
- Criteres "c'est reussi"
- Planning suggere (demi-journee)
- Petite histoire
- Erreur classique
- En vrai
- A toi
- Zoom : demi-journee, vrai contrat

13. Chapitre 13 - A retenir

- Le flux et les branches
- Rebase, merge, historique
- Revue, protection, CI
- Tags, cherry-pick, bisect, secrets
- Petite histoire
- Erreur classique
- En vrai
- A toi

14. Chapitre 14 - Atelier : flux d'equipe

- But et materiel
- Etapas
- Petite histoire
- Criteres et blocages
- Erreur classique
- En vrai
- A toi

15. Chapitre 15 - Atelier : revue de code

- But et preparation
- Etapas auteur
- Etapas reviewer
- Etapas retour auteur
- Petite histoire
- Criteres et pieges
- Erreur classique
- En vrai
- A toi

16. Chapitre 16 - Atelier : conflit en equipe

- But et scenario
- Etapas
- Variante rebase
- Petite histoire

- Criteres et panique
- Erreur classique
- En vrai
- A toi

17. Chapitre 17 - Fork et upstream : contribuer ailleurs

- Les mots
- Scenario concret
- Les commandes d'installation
- Rester a jour avec upstream
- Le flux de contribution
- Bonnes manieres
- Et dans une entreprise ?
- Petite histoire
- Erreur classique
- En vrai
- A toi

18. Chapitre 18 - Hygiene des secrets : jamais de clés dans Git

- Ce qu'on ne commit pas
- .gitignore est ton ami
- Comment l'equipe partage les secrets
- Si le mal est fait
- Exemple Max
- Checklist avant le premier push d'une feature API
- Petite histoire
- Erreur classique
- En vrai
- A toi

19. Chapitre 19 - Bonnes pratiques d'equipe

- Ecrire le contrat léger
- Petites PR, souvent
- Synchroniser tot
- Protéger ce qui compte
- Review comme un collègue, pas comme un juge
- Noms clairs partout
- Une intention à la fois
- Sabbat du force-push
- Documenter les incidents sans humiliation
- Rituels hebdomadaires légers
- Petite histoire
- Erreur classique
- En vrai
- A toi

20. Chapitre 20 - Quiz final

- Questions
- Reponses
- Score
- Petite histoire
- Erreur classique
- En vrai
- A toi

21. Chapitre 21 - Bravo

- Petite histoire
- Ce que tu peux faire maintenant
- Erreur classique
- En vrai
- Mission
- A toi
- Petite scene finale

Chapitre 1 - Rappel rapide et carte du livre



Après les bases : flux d'équipe, revue, CI, releases.

Tu as déjà fait les bases. Init, clone, status, add, commit, log, branch, merge, pull, push, stash, undo, et une première pull request. Si un détail est flou, ce n'est pas grave : on ne va pas tout refaire. Ce livre part du moment où Git devient un outil d'**équipe**, pas un carnet personnel. Chez DanielCraft, on aime une image nette : les bases, c'était apprendre à conduire seul sur un parking. Ici, c'est la route à plusieurs. Se synchroniser, se revoir le code, protéger la branche principale, sortir une version sans panique.

Tu sais créer un **commit**, créer une **branche**, fusionner, pousser et tirer depuis GitHub ou un équivalent. Tu as vu un conflit une fois. C'est assez pour avancer. On ne revient pas sur `git init`, ni sur "c'est quoi un commit", ni sur l'installation. Si tu bloques sur ces mots, relis le premier livre Git. Ici, on monte d'un cran : travailler à deux, trois ou quatre personnes sans se marcher dessus.

Ce que ce n'est pas

Ce n'est pas un dictionnaire exhaustif de Git. Ce n'est pas non plus un cours DevOps pour cent personnes. Ce n'est pas "apprendre encore une fois ce que tu sais déjà". Et ce n'est surtout pas "connaître Git" confondu avec "savoir collaborer". Les bases sont le moteur. L'équipe, c'est le code de la route : priorités, reviews, **branches protégées**. Sans ça, tout le monde pousse sur `main` et un jour quelqu'un casse la prod un vendredi soir.

Ce n'est pas non plus un livre où l'on te juge si tu as oublié une option obscure. On veut des gestes tenables. Petit. Fréquent. Clair.

Imagine une petite équipe qui livre un site vitrine avec formulaire de contact. Pour que ça marche vraiment, plusieurs pièces s'emboîtent. D'abord un **flux** clair : qui pousse ou, quand on tire, comment on se parle. Ensuite une stratégie de branches simple, souvent feature vers main. Rebase et merge sans dogme : l'idée, les risques, le moment. Un historique propre avec des messages qui racontent le pourquoi. La revue de code humaine, bienveillante. Les branches protégées. Une CI légère sur les pull requests. Les tags et releases. Puis cherry-pick et bisect, deux outils pour reprendre un commit et trouver celui qui a cassé.

★ A RETENIR

Connaître Git, ce n'est pas encore savoir collaborer. Le flux d'équipe, c'est le vrai cran suivant.

Lea (front), Max (back) et Sam (fullstack) reviendront souvent. Ils sont trois sur ce petit site. Parfois on ajoute un bug urgent ou une release. Des situations concretes, pas un manuel militaire. Tu verras le meme geste sous plusieurs angles : partir d'une branche a jour, ouvrir une PR propre, repondre a une review, ne jamais pousser directement sur `main` sans filet.

Ce que tu vas savoir faire

Dans ce livre, tu vas poser un flux d'equipe tenable, choisir une strategie de branches simple, comprendre rebase vs merge sans religion, garder un historique lisible, faire des revues utiles, proteger `main`, ajouter une CI legere, taguer des releases, cherry-pick en urgence, bisecter un bug, simuler l'equipe en mini-projet, contribuer via fork, et garder les secrets hors du depot. Puis un recap, trois ateliers, les bonnes pratiques, un quiz, et un bravo.

Niveau debutant solide qui a deja les bases Git. Pas besoin d'etre expert DevOps. Besoin de curiosite et d'honnete : Git aide a collaborer ; il ne remplace pas la communication.

Comment lire ce livre

Lis dans l'ordre au debut. Les premiers chapitres posent le sol. Les ateliers font faire. Le quiz verifie. Tu peux revenir ensuite a un chapitre precis (rebase, revue, secrets) comme a une fiche. A chaque fin de chapitre, il y a un "A toi". Fais-le. Cinq minutes valent mieux qu'une lecture passive de quarante pages. Tape les commandes sur un depot de test, pas sur le client critique.

Petite histoire

Lea avait "deja Git" sur son CV. Le premier jour en equipe, Max lui a dit : "Tu as pousse sur main ?" Elle avait oublie la regle. Sam a du revert un vendredi. Personne n'avait ecrit le flux. Ce livre existe pour que cette scene ne se repete pas chez toi. Trois personnes, un README, un filet technique, et des vendredis plus calmes.

Chez DanielCraft, on a vu la meme scene trop souvent : talent technique, zero contrat leger. Le code etait bon. Le vendredi, non.

Erreur classique

Croire que "je connais Git" egal "je sais collaborer". Ou croire que "on est que deux, on peut tout faire sur main". A deux, un revert malheureux, et vous avez le meme probleme qu'a dix. La discipline legere protege aussi les petites equipes. Autre piege : lire ce livre sans depot de test. Tu retiens les mots, pas les gestes.

⚠ ATTENTION

Sans depot de test, tu monologues. Avec un depot de test, tu t'entraines. Cree-en un avant le chapitre 2.

En vrai

Ouvre un vieux depot ou tu as travaille a plusieurs. Note ce qui a frotte : conflits, messages flous, PR trop grosses, `main` non protegee, un secret qui a failli partir. Ce livre repond a ca. Si tu n'as pas de depot, cree-en un vide sur GitHub. Git installe, un terminal, idealement un second compte ou un ami pour jouer la revue. Sinon, simule seul en changeant de branche et en te commentant toi-meme.

A toi

Ecris en trois phrases la regle d'or que tu voudrais dans ton equipe. Exemple : "jamais de push direct sur main", "toute feature passe par une PR", "on tire main avant de creer une branche". Garde ce but. On y reviendra au mini-projet et dans le README.

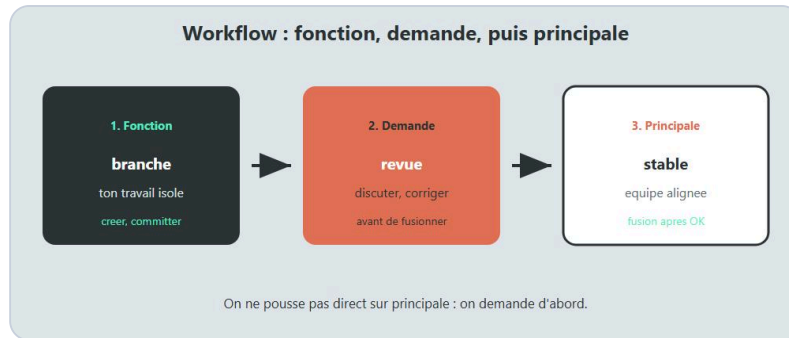
Petite scene DanielCraft

Lea ouvre le README. Max ajoute une ligne sur les PR. Sam active la protection de `main` sur le depot de test. Trois gestes, une meme posture : on se protege avant d'aller vite. C'est exactement l'esprit de ce livre.

♦ ASTUCE

Avant le chapitre 2, cree un depot de test vide et invite un ami (ou un second compte) pour jouer la revue. Le livre devient concret des la page suivante.

Chapitre 2 - Flux d'equipe : qui pousse ou, quand tirer



Pull, branche, commit, push, revue : le rythme d'equipe.

Git seul, c'est un carnet. Git en equipe, c'est un carnet partage ou tout le monde ecrit en meme temps. Sans accord, c'est le chaos poli : "j'ai pousse sur main", "j'ai ecrase ta branche", "je n'ai pas tire depuis trois jours". Ce chapitre pose le **flux**. Pas la theorie des remotes. Le rythme du travail a deux ou trois personnes sur un meme site. Chez DanielCraft, on imagine souvent un tableau blanc au milieu de la piece. **main** sur le serveur, c'est le tableau officiel. Ta machine a une copie. La machine de Lea aussi. Si chacun dessine sans regarder le tableau, vous vous ecrasez. Le flux ecrit bat le flux imagine. Une demi-page dans le README vaut mieux qu'une reunion d'une heure tous les mois pour "reclarifier".

Le flux, c'est la regle : je mets a jour ma copie, je dessine sur mon coin (ma **branche**), je propose mon dessin (**PR**), quelqu'un regarde, puis on colle sur le tableau. Petit et frequent bat grand et rare. Une feature qui vit deux jours, avec quatre petits commits et une PR de cent vingt lignes, se review facilement. Une branche qui vit trois semaines, avec quarante commits melanges et deux mille lignes, devient un roman. Personne ne le lit vraiment. On clique Approve en esperant.

★ A RETENIR

Petit et frequent bat grand et rare. Une PR lisible vaut mieux qu'une epique ignoree.

Une journee type a trois

Le matin, Max arrive. Il veut corriger un bug sur le formulaire. Il fait ceci, dans cet esprit :

```
git switch main
git pull
git switch -c fix/formulaire-email
```

Il travaille. Il commit. Il pousse sa branche, pas **main** :

```
git push -u origin fix/formulaire-email
```

Il ouvre une pull request. Lea regarde. Sam teste le formulaire. On merge. Max revient sur **main** et tire :

```
git switch main
git pull
```

Lea, pendant ce temps, travaille sur `feature/page-tarifs`. Elle a tiré `main` ce matin. Elle n'a pas besoin d'attendre Max pour coder. Elle aura besoin de se resynchroniser avant de merger, surtout si Max a touché des fichiers proches. En petite équipe, la règle simple est : chacun pousse sur sa branche feature ou fix, personne ne pousse directement sur `main`. Le dépôt distant (`origin`) est la mémoire commune. Ta machine n'est pas la source de vérité pour les autres.

Quand tirer (pull)

Tire `main` au début de ta session. Tire `main` avant d'ouvrir une PR. Tire `main` si quelqu'un vient de merger quelque chose d'important pendant que tu codes. Tu n'as pas besoin de tirer toutes les cinq minutes. Tu as besoin de tirer avant les moments où ça compte : démarrer, synchroniser, livrer.

```
git switch main
git pull
git switch feature/page-tarifs
git merge main
```

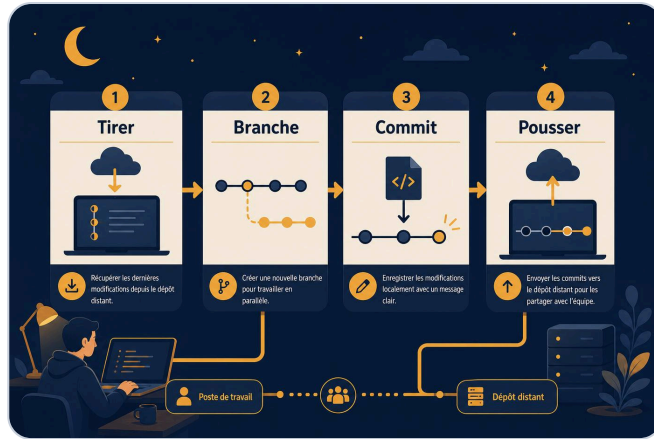
Ici, Lea ramène les nouveautés de `main` dans sa feature. Elle peut aussi rebase (chapitre 4). L'intention est la même : ne pas découvrir le conflit le jour du merge final. Git ne remplace pas un message. "Je touche `contact.html` aujourd'hui" évite deux personnes sur le même fichier. "PR prête, besoin d'un regard" évite que la PR pourrisse trois jours. Dans une équipe de trois, un canal suffit. Pas besoin d'un processus de douze pages. Besoin de dire ce qu'on fait.

♦ ASTUCE

Un message dans le canal coûte dix secondes. Un conflit surprise sur le même fichier coûte une heure. Parle avant de tirer.

Le rythme d'une journée tient en quelques gestes. Le matin : tire `main`, choisis ta tâche, crée ou reprends ta branche. Midi ou en fin de tranche : pousse ta branche pour ne pas garder le travail seul sur ton disque. Avant la pause longue : status propre ou stash conscient. En fin de feature : PR plus message dans le canal. Après merge : retour sur `main`, pull, suppression de branche. `index.html`, la config, le schéma de base : si deux personnes y touchent le même jour, parlez-vous. Git fusionnera peut-être. Le sens métier, lui, demandera un humain.

Un message coûte dix secondes. `git pull` ramène ce qui est déjà poussé. Si Lea n'a pas poussé, tu ne vois pas son travail. Si tu n'as pas poussé, personne ne voit le tien. La mémoire commune, c'est le serveur, pas ton disque.



Exemple : le parcours en quatre gestes.

Petite histoire

Sam attend la PR de Max pour avancer. Max est en réunion. Sam continue sur une autre tâche au lieu d'"emprunter" la branche de Max pour y pousser sans prévenir. Surprise égal conflits plus egos. Le lendemain, Max merge. Sam tire `main`, reprend. Quatre messages dans le canal, zero drame. C'est ça, le flux : le travail utile finit sur le serveur, sur une branche claire, avec des humains qui se disent ou ils vont.

Chez DanielCraft, on a vu l'inverse trop souvent : "j'ai emprunté ta branche, j'ai forcé, ça marche chez moi". Deux heures de réparation. Le flux écrit aurait suffi.

Erreur classique

Coder deux jours sur `main` en local, puis faire un gros push. Ou ne jamais tirer et découvrir quinze commits des autres au moment de merger. Ou pousser `--force` sur une branche partagée "parce que ça marche chez moi". Autre piège : croire que "on est que deux, on peut tout faire sur main". A deux, un vendredi soir, un revert malheureux, et vous avez le même problème qu'à dix.

⚠ ATTENTION

`--force` sur une branche partagée, c'est souvent un écrasement poli. Préfère `--force-with-lease`, et seulement sur ta branche perso.

En vrai

Demain matin, avant de coder, fais le rituel :

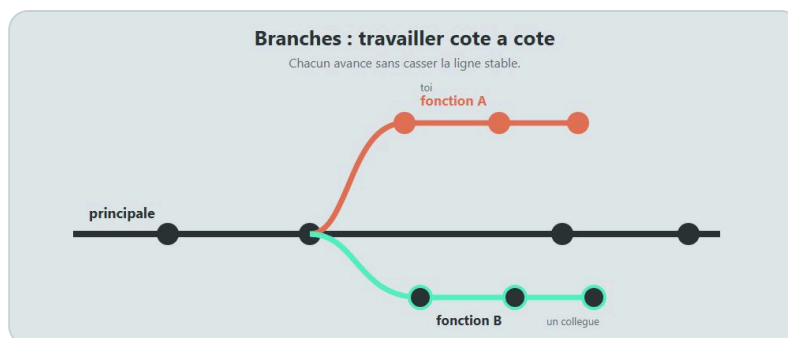
```
git switch main
git pull
git status
```

Puis crée ou reprends ta branche. Note combien de fois cette semaine tu as tiré `main`. Si la réponse est "une fois lundi", augmente le rythme.

A toi

Ecris le flux de ton equipe en cinq lignes maximum, comme une recette. Exemple : "1) pull main 2) branche 3) commits 4) push branche 5) PR 6) review 7) merge 8) pull main". Colle-le dans le README du depot de test. Tout le monde doit pouvoir le lire en trente secondes.

Chapitre 3 - Strategie de branches simple (main + feature)



main stable + branches de travail courtes.

Une branche, tu la connais déjà : une ligne d'historique parallele. En equipe, la question n'est plus "comment creer une branche", c'est "quelles branches on garde, et pour quoi faire". Si tout le monde invente ses noms et ses durees, le depot devient un grenier. Si vous vous mettez d'accord une fois, Git devient invisible : il porte le travail sans drama. Chez DanielCraft, on recommande pour deux a cinq personnes le modele **main** plus **features**. Clair. Suffisant. Facile a expliquer a un stagiaire le premier jour. On l'appelle parfois **GitHub Flow** en version legere : une branche courte, une PR, un merge, on recommence. Pas besoin de six types de branches pour livrer un site vitrine. Besoin d'une intention claire et d'une duree courte.

main (parfois encore **master** ; on prefere **main**) est la branche de reference. Elle doit etre deployable, ou au moins "connue bonne". Chaque nouvelle idee vit sur une branche feature ou fix :

```
git switch main
git pull
git switch -c feature/page-prix
```

Quand c'est pret et review : merge dans **main** via **pull request**. Puis on efface la branche feature. Elle a servi. Elle n'a pas vocation a vivre des mois.

★ A RETENIR

Une branche feature a une intention, un nom clair, et une duree courte. Apres merge, elle disparaît.

Noms et duree de vie

Un bon nom dit l'intention. **feature/page-prix**, **fix/login-timeout**, **chore/maj-dependances**. Tu peux utiliser **feature/** pour une nouveaute, **fix/** pour un correctif, **chore/** pour du menage, **docs/** pour la doc. Ce n'est pas une loi universelle. C'est une convention d'equipe. Choisissez-en une et tenez-la. Evitez **max-wip**, **test2**, **final-final-v3**. Dans six mois, personne ne saura ce que c'etait. Y compris Max.

Idealement, une branche vit heures ou quelques jours. Pas trois semaines sans sync. Plus une branche vit longtemps, plus **main** avance sans elle, plus le retour sera douloureux. Synchronise souvent :

```
git switch main
git pull
git switch feature/page-prix
git merge main
```

Ou avec rebase (chapitre suivant). L'idée : ramener régulièrement les nouveautés de `main` dans ta feature, avant le gros conflit du vendredi.

♦ ASTUCE

Si ta branche a plus d'une semaine, synchronise `main` dedans aujourd'hui. Pas "quand j'aurai le temps".

Ce que ce n'est pas (encore)

GitFlow avec `develop`, `release/`, `hotfix/` existe. Il aide les grosses organisations avec des calendriers de release lourds. Pour une petite équipe produit qui déploie souvent, c'est souvent trop de cérémonie. Trunk-based development (tout le monde merge très souvent dans `main`, parfois avec feature flags) est une autre école. Puissant, mais demande discipline et souvent une CI solide. Pour ce livre : maîtrise d'abord `main` plus feature plus PR. Tu pourras complexifier plus tard si le besoin est réel, pas par snobisme.

Si Lea corrige un bug login et ajoute une page prix dans la même branche, la review mélange deux sujets. Au merge, si la page prix est douteuse mais le bug urgent, vous êtes bloqués. Deux branches. Deux PR. Le bug peut partir ce soir. La page prix demain après discussion. Un bug en production : Sam crée `fix/bouton-payer` depuis `main` à jour, corrige, ouvre une PR courte, fait merger vite, déploie. Même modèle que la feature. Juste plus urgent, donc plus petit, plus cible. Ne crée pas une usine à gaz "hotfix process" à trois personnes.

Après merge, supprime la branche locale et distante. GitHub a souvent une case "Delete branch" après merge. Localement :

```
git switch main
git pull
git branch -d feature/page-prix
```

De temps en temps, liste les vieilles branches. Un dépôt avec quatre-vingts branches mortes fatigue tout le monde. Jour 1, tu peux coller au mur : "1. Je tire `main`. 2. Je crée `feature/ou-fix/quelque-chose`. 3. Je commit. 4. Je pousse ma branche. 5. J'ouvre une PR. 6. On review. 7. On merge. 8. Je retiens `main` et j'efface ma branche."

Petite histoire

Max a créé `feature/mega-refonte` et y a vécu trois mois. Le merge a été un vendredi noir. Lea et Sam ont passé le week-end à résoudre des conflits mélangés avec trois sujets différents. Depuis, l'équipe a écrit : "Les branches feature vivent moins d'une semaine si possible." Une règle écrite bat une règle imaginée.

Chez DanielCraft, on raconte souvent cette scène pour calmer les ambitions de branche éternelle. L'ambition, c'est bien. La tranche livrable, c'est mieux.

Erreur classique

Creer `feature/mega-refonte` et y vivre trois mois. Ou committer directement sur `main` "juste pour un petit truc". Les petits trucs s'accumulent. Puis un jour `main` est cassee et personne ne sait quel petit truc a fait le mal. Autre piege : dix branches `feature/tmp` sans description. Le grenier gagne. Parfois deux personnes travaillent sur la meme feature : communication forte, petits commits, pull frequents. Evite que chacun reecrive l'historique de l'autre sans se parler.

⚠ ATTENTION

Un "petit truc" sur `main` sans PR, c'est souvent le debut d'une chasse au bug sans filet. Passe par une branche, meme pour dix lignes.

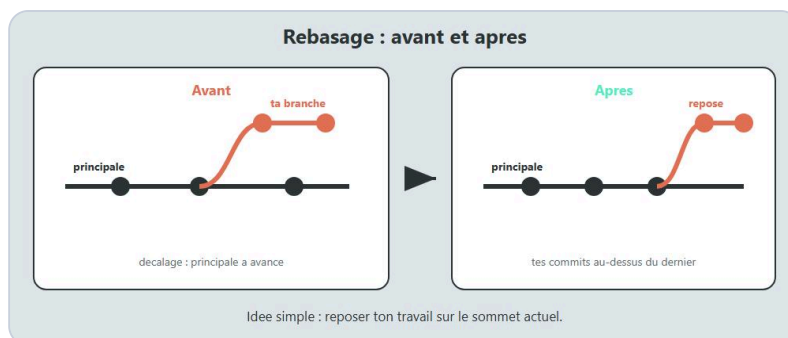
En vrai

Ouvre ton depot. Liste les branches avec `git branch -a`. Combien sont mortes ? Combien ont un nom obscur ? Note trois renommages ou suppressions a faire. Hygiene egal clarte.

A toi

Ecris dans le README : "On travaille avec main plus feature/*. Toute integration passe par une PR. Les branches feature vivent moins d'une semaine si possible." Adapte si besoin, mais écris quelque chose. Une regle ecrite bat une regle imaginee.

Chapitre 4 - Rebase vs merge : idee, quand, risques



Rebase ou merge : deux facons de rejoindre l'histoire.

Tu as deja fusionne avec `git merge`. Ca cree parfois un commit de merge, et l'historique montre que deux lignes se sont rejointes. Le **rebase**, c'est une autre facon de "mettre a jour" ta branche : rejouer tes commits comme s'ils etaient partis plus tard, sur une base plus recente. Ce n'est pas une religion. Chez DanielCraft, on refuse le dogme "toujours rebase" ou "jamais rebase". On veut que tu comprennes l'idee, le moment, et le danger. Un outil mal compris fait plus de degats qu'un outil "moins elegant" bien tenu.

Imagine que `main` avance. Ta feature est partie d'un vieux point. **Merge**, c'est coller les deux histoires avec un noeud. Rebase, c'est decouper tes commits de ta feature et les recoller proprement au bout de `main`, un par un. Resultat frequent du rebase : un historique plus lineaire, plus facile a lire avec `git log --oneline`. Resultat frequent du merge : un historique qui montre vraiment les allers-retours d'equipe. Les deux sont valides. Le choix depend du contexte et de l'accord d'equipe.

★ A RETENIR

Merge et rebase sont deux outils. Ni honte, ni heroisme. Choisis, ecris la politique, respecte-la.

Un cas concret : Lea met a jour sa feature

Lea est sur `feature/page-tarifs`. Max a merge un fix sur `main`. Lea veut ces changements avant sa PR. Option merge :

```
git switch feature/page-tarifs
git fetch origin
git merge origin/main
```

Option rebase :

```
git switch feature/page-tarifs
git fetch origin
git rebase origin/main
```

Dans les deux cas, elle peut avoir des conflits. Elle les regle. La difference, c'est la forme de l'historique ensuite. Le rebase aide souvent sur une branche feature personnelle, pas encore partagee, pour la "reposer" sur `main` avant la review. Il aide aussi pour nettoyer localement avant de pousser, parfois avec un rebase interactif. Attention : interactif egal plus de puissance, plus de responsabilite.

Quand le merge est plus simple (et plus sur)

Si plusieurs personnes poussent sur la meme branche feature, un rebase plus force push peut casser le travail des autres. Merge est alors plus doux : il n'exige pas de reecrire l'historique deja publie. Si tu n'es pas a l'aise, merge. Un merge un peu bruyant bat un rebase mal compris qui reecrit l'histoire de tout le monde. Regle d'or : ne rebase pas `main` . Ne force-push pas `main` . Ne rebase pas la branche d'un collegue sans qu'il le sache.

```
# Sur TA feature, apres rebase local, si deja poussee :  
git push --force-with-lease
```

`--force-with-lease` est plus poli que `--force` : il refuse d'ecraser si quelqu'un a pousse entre-temps. Prefere-le. Que tu merges ou rebases, un **conflit** veut dire : deux changements touchent la meme zone. Tu ouvres le fichier, tu choisis ou combines, tu marques resolu. En merge : `git add fichier.html` puis `git merge --continue` . En rebase : `git add fichier.html` puis `git rebase --continue` . Si tu es perdu : `git rebase --abort` . Tu reviens a l'etat d'avant. Respire.

⚠ ATTENTION

Ne rebase jamais `main` partagee. Ne force-push jamais `main` . Sur une feature perso deja poussee, prefere `--force-with-lease` .

Exemple de politique simple pour une equipe de trois : on merge les pull requests dans `main` (bouton Merge sur GitHub, parfois "squash merge" - chapitre 5). Sur sa feature perso, chacun peut rebase sur `main` avant la PR pour limiter le bruit. Personne ne rebase une branche partagee sans message dans le canal. Ecrivez deux phrases dans le README. Ca suffit. L'outil sert l'equipe, pas l'inverse.

Sam explique souvent le graphique : merge = noeud visible, rebase = ligne droite. Lea prefere la ligne droite sur sa feature. Max prefere merge partout "parce que je dors mieux". Les deux politiques marchent si elles sont ecrites.

Petite histoire

Sam a explique a un stagiaire que merge etait "moche". Le stagiaire a eu peur de merge pendant un mois. Puis il a fait un rebase force sur une branche partagee. Deux heures de reparations. Chez DanielCraft, on dit : merge et rebase sont deux outils. Ni honte, ni heroisme. Choisis, ecris, respecte.

Lea, elle, a teste les deux sur un depot de jouet. Elle a regarde `git log --graph` . Elle a choisi. Depuis, plus de dogme silencieux dans l'equipe.

Erreur classique

Faire un rebase sur `main` local puis pousser en force "parce que mon log est plus joli". Ou lancer un rebase interactif le jour de la release sans filet. Ou expliquer que merge est "moche" : tu crees de la honte inutile. Le pire choix, c'est le non-choix ou chacun fait sa religion en silence.

♦ ASTUCE

Si tu es perdu en plein rebase : `git rebase --abort` . Tu reviens a l'avant. Puis tu respireras, puis tu recommenceras.

En vrai

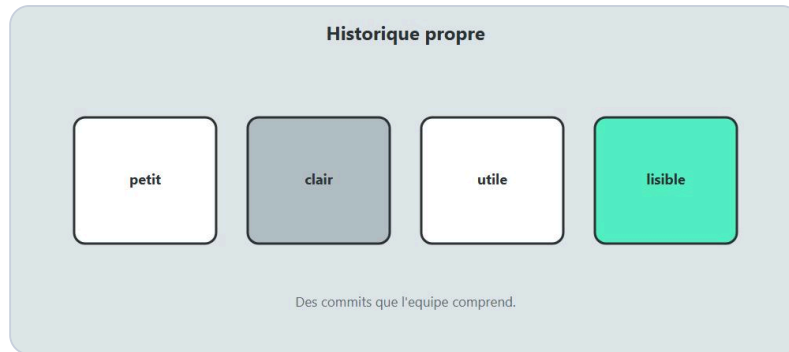
Sur un depot de test, cree une branche, fais deux commits, avance `main` avec un autre commit, puis teste les deux options (merge puis, dans une autre copie de branche, rebase). Regarde la difference avec tes yeux :

```
git log --oneline --graph --decorate -15
```

A toi

Choisis avec ton equipe (meme si l'equipe c'est toi et un ami) : "rebase OK sur feature perso, merge pour integrer dans main" ou "merge partout pour simplifier". Ecris le choix. Le pire choix, c'est le non-choix.

Chapitre 5 - Historique propre : messages, petits commits, squash



Des commits petits, clairs, lisibles.

L'historique Git, c'est le journal de bord du projet. Dans six mois, Lea ouvrira `git log` pour comprendre pourquoi le formulaire valide l'email d'une certaine façon. Si elle lit "fix", "wip", "asdf", "final", elle perdra du temps. Si elle lit des messages clairs et des commits digérés, elle avancera. Un **historique** propre n'est pas de la vanité. C'est de la gentillesse envers le futur, toi y compris. Chez DanielCraft, on préfère un ton simple, factuel, sans emoji obligatoire, sans roman policier.

Un bon **message** dit surtout l'intention. "Corrige le timeout du login" bat "modif fichier". "Ajoute la section tarifs sur la page offres" bat "update". Tu n'as pas besoin d'un roman. Une ligne claire suffit souvent. Si le changement est subtil, une deuxième ligne après une ligne vide aide :

```
Corrige la validation email du formulaire contact

Le champ acceptait les adresses sans point dans le domaine.
```

Un commit égal une intention cohérente. Tu peux avoir plusieurs commits dans une PR, tant que chaque commit raconte une étape logique : structure HTML, puis styles, puis texte. Évite le commit unique de quatre-vingts fichiers "toute la feature" si tu peux découper sans souffrir. Inversement, évite quarante commits "typo", "typo2", "aaa", "wip" laissés tels quels sur `main`. Avant la merge, tu peux nettoyer (**squash**, ou rebase interactif si tu maîtrises).

★ A RETENIR

Le message répond au pourquoi. Les fichiers, Git les montre déjà dans le diff.

L'idée du squash

Squash égal compresser plusieurs commits en un (ou en peu). Sur GitHub, l'option "Squash and merge" prend tous les commits de la PR et les intègre dans `main` comme un seul commit. Pratique pour garder `main` lisible quand la feature a vécu avec des allers-retours. Ce n'est pas obligatoire. Certaines équipes aiment garder tous les commits de la PR. D'autres squashent toujours. Choisissez. L'important est que `main` reste navigable.

Tu viens de commit et tu as oublié un fichier, ou une typo dans le message. Si tu n'as pas encore poussé (ou si tu es seul sur la branche et d'accord pour réécrire) :

```
git add fichier-oublie.css
git commit --amend --no-edit
```

N'amends pas un commit déjà sur `main` partagé. N'amends pas le travail d'un autre. **Amend** égal micro-rewrite. Puissant, local, dangereux si mal place. Un historique "propre" commence aussi par ne pas y coller des secrets. Les mots de passe, les clés API, les dumps de base (chapitre 18). Une fois poussé, un secret est difficile à vraiment effacer.

⚠ ATTENTION

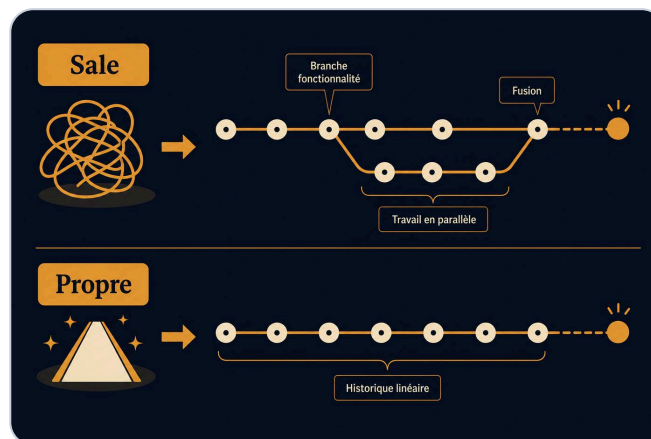
`commit --amend` sur un commit déjà publié et partagé, c'est réécrire l'histoire des autres. Réserve-le à ta branche perso non partagée, ou annonce et utilise `--force-with-lease`.

Avant d'ouvrir la PR, regarde :

```
git log main..HEAD --oneline
git diff main...HEAD
```

Est-ce que la liste des commits raconte une histoire ? Est-ce qu'il y a un fichier de debug à retirer ? Un `console.log` oublié ? Un commentaire "TODO enlever" qui devait partir ? Cinq minutes ici épargnent vingt minutes de review grevée. Tu verras parfois `feat:`, `fix:`, `docs:` (conventional commits). Utile si l'équipe s'en sert pour des changelogs automatiques. Pas obligatoire. Mieux vaut un message clair sans préfixe qu'un préfixe vide de sens.

Lea lit le log à voix haute avant d'ouvrir la PR. Si elle sourit nerveusement, elle réécrit. Max préfère des commits déjà propres. Sam squash via GitHub au merge. Trois styles, une même exigence : `main` lisible.



Exemple : passer d'un historique sale à un propre.

Petite histoire

Lea a six commits sur sa feature tarifs : "wip", "wip2", "styles", "fix typo", "ok", "ok final". Avant d'ouvrir la PR, elle regarde le log et se dit que Sam va souffrir. Elle décide de squash via le bouton GitHub au merge, et elle réécrit le message final : "Ajoute la section tarifs sur la page offres". `main` reste lisible. Sa branche feature peut garder ses allers-retours : ce n'est qu'un brouillon. Max, lui, préfère des commits déjà propres avant la PR. Les deux approches marchent si l'équipe est d'accord sur ce que doit ressembler `main`.

Chez DanielCraft, on dit souvent : le brouillon peut etre sale ; la page officielle, non.

Erreur classique

Honte du "mauvais historique" au point de ne plus oser commit. Commit souvent en local sur ta feature. Nettoie avant d'integrer si besoin. L'inverse aussi : pousser un journal de bord illegible sur `main` parce que "on verra plus tard". Plus tard, c'est Lea un mardi a 18h. Sur ta branche perso non partagee, "wip" en local le temps d'un cafe, pourquoi pas. Avant la review, nettoie ou squash. Laisser "wip" sur `main` apres merge, c'est offrir une enigme a toute l'equipe.

♦ ASTUCE

Avant la PR : `git log main..HEAD --oneline` puis `git diff main...HEAD` . Cinq minutes de lecture epargnent vingt minutes de review.

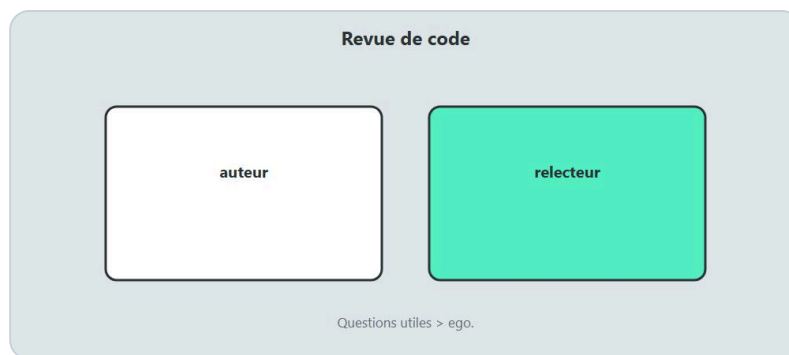
En vrai

Prends une ancienne PR (ou un vieux `git log`). Note trois messages obscurs. Reecris-les comme tu aurais voulu les lire. Cet exercice calibre ton prochain commit. Imagine que tu ne connais pas le projet. Est-ce que le message plus le diff racontent une histoire ? Si tu dois expliquer oralement pendant cinq minutes, le message est trop maigre.

A toi

Pour ta prochaine branche, impose-toi cette regle : chaque message repond a "pourquoi ce changement existe". Pas "quels fichiers". Le pourquoi. Puis ouvre le log et lis-le a voix haute. Si tu souris nerveusement, recommence le message.

Chapitre 6 - Revue de code humaine



Auteur et relecteur : questions utiles.

La **pull request** n'est pas un portail magique. C'est une conversation. Quelqu'un propose un changement. Quelqu'un d'autre regarde, pose des questions, suggère, approuve. Git transporte les diffs. Les humains portent le sens. Sans revue, on merge des bombes polies. Avec une revue toxique, on merge moins, on cache, on a peur. L'objectif : une **revue** utile et bienveillante. Chez DanielCraft, la revue est un filet, pas un tribunal.

Parce que quatre yeux voient plus que deux. Parce que Max connaît le back et peut voir que le front de Lea casse une API. Parce que Sam se souvient qu'un cas mobile n'est pas testé. Parce qu'écrire pour être lu oblige à clarifier. Tu n'as pas à être un compilateur humain. Priorise : est-ce que la PR fait ce qu'elle dit ? Est-ce que le **perimetre** est raisonnable (pas trente sujets) ? Est-ce qu'il y a un risque sécurité évident (secret, injection grossière, permission trop large) ? Est-ce que les noms aident ? Est-ce que tu saurais maintenir ce code dans trois mois ? Est-ce que l'auteur a indiqué comment tester ?

★ A RETENIR

La revue aide. Elle ne juge pas. Dire aussi ce qui est clair change toute l'ambiance.

Le ton qui aide

Préfère "Ici, si l'email est vide, on n'affiche pas d'erreur - on pourrait bloquer l'envoi" à "C'est nul". Préfère "Je ne comprends pas cette condition, tu peux m'expliquer ?" à "Lol quoi". Préfère "Suggestion : " à "Obligatoire : " quand c'est vraiment une préférence. Dis aussi ce qui est bien. "Claire la séparation HTML / CSS" prend trois secondes et donne de l'énergie. La revue n'est pas seulement une chasse aux défauts.

En tant qu'auteur, aide le reviewer. Titre clair. Description courte : but, changements principaux, comment tester. Capture d'écran si c'est visuel. Lien vers une issue si elle existe :

```

## But
Afficher les tarifs sur /offres

## Changements
- section tarifs dans offres.html
- styles associes
- liens depuis l'accueil

## Comment tester
1. Ouvrir /offres
2. Verifier mobile
3. Cliquer "Contact" depuis un tarif

```

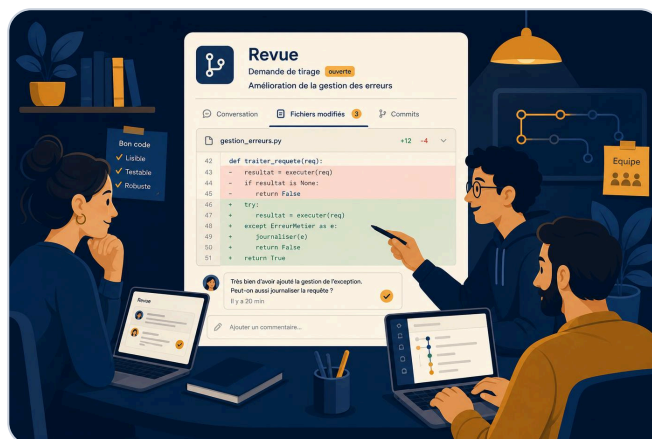
Reponds aux commentaires sans te defendre par principe. Si tu n'es pas d'accord, explique calmement. Si tu corriges, pousse un commit et dis "c'est a jour". Une PR de cent cinquante lignes se review. Une PR de deux mille cinq cents lignes se feint. Decoupe. Si tu ne peux pas decouper, guide le reviewer : "commence par `api.js`, puis le template". En petite equipe, vise un premier regard en vingt-quatre heures quand c'est possible. Meme un "je regarde demain matin" compte.

♦ ASTUCE

"Comment tester" dans la description, c'est un cadeau au reviewer. Trois etapes suffisent souvent.

Sam sur la PR de Lea : "J'ai suivi tes etapes de test sur mobile : le bloc tarifs passe bien. Une question : sur tres petit ecran, le prix passe sous le bouton - voulu ? Si non, on peut empiler en colonne des 480px. Suggestion, pas bloqueur si tu vises desktop d'abord." Lea repond : "Pas voulu. Je pousse un correctif ce soir." Elle pousse. Sam Approuve. Merge. Quatre messages, zero ego, un meilleur site. Sur GitHub, tu peux commenter sans bloquer, demander des changements, ou approuver. Utilise "request changes" pour les vrais blocants. N'approuve pas par pitie si tu n'as pas regarde.

Max, un jour, a Approuve "parce que c'etait petit" sans ouvrir les fichiers. Le formulaire a casse. Depuis, l'equipe dit : "Approve conscient ou pas Approve." Ce n'est pas de la rigidite. C'est du respect pour la prod et pour Lea qui devra debugger.



Exemple : une revue d'equipe sur une PR.

Petite histoire

Max a approuve une PR "parce que c'était petit" sans ouvrir les fichiers. Le formulaire a cassé la prod le vendredi. Depuis, l'équipe dit : "Approve conscient ou pas Approve." Lea, elle, a appris à écrire "comment tester" avant chaque PR. Les reviews sont devenues plus rapides parce qu'elles sont plus claires. Ce n'est pas de la triche. C'est de la politesse technique.

Chez DanielCraft, on répète : le ton compte autant que le fond. Une remarque juste dite mal, et la prochaine PR sera cachée. Une remarque moyenne dite bien, et l'auteur corrige avec le sourire.

Erreur classique

La revue ego : "moi j'aurais fait autrement" sur chaque ligne. Ou la revue fantôme : Approve sans ouvrir les fichiers. Ou l'auteur invisible qui n'écrit aucune description. Les trois cassent le jeu. Tu peux quand même aider sans être expert du domaine : clarté des noms, cas limites évidents, secrets, "je ne comprends pas ce bloc". Dire "je ne connais pas bien cette API, Max peut-tu jeter un œil ?" est professionnel.

⚠ ATTENTION

Approve fantôme et Request changes pour une virgule : deux extrêmes qui cassent la confiance. Vis le fond, garde le ton.

En vrai

Prends une PR passée (la tienne ou une publique). Écris trois commentaires bienveillants et utiles que tu aurais pu laisser. Puis écris le commentaire toxique équivalent... et jette-le. Sens la différence. Garde la première version comme modèle mental.

A toi

Ajoute dans le README de l'équipe trois phrases : "On review pour aider.", "On explique le comment tester.", "On dit aussi ce qui est clair." Lis-les avant ta prochaine review.

Chapitre 7 - Branches protegees : pas de push direct sur main



main protegee : pas de push direct.

Tu peux ecrire la belle regle "on ne pousse pas sur main" dans le README. Un jour, quelqu'un oubliera. Un autre jour, un stagiaire ne l'aura pas lue. Un vendredi, tu pousseras toi-meme "juste un hotfix" a 19h. Les **branches protegees**, c'est le filet technique. GitHub (et GitLab, et d'autres) peuvent refuser le push direct sur `main` et imposer le passage par une **pull request**. Chez DanielCraft, on prefere un message d'erreur clair qu'une prod cassee.

Protger `main`, c'est dire au serveur : "cette branche est speciale". On n'y ecrit pas comme sur un brouillon. On y arrive apres **review** (et souvent apres des checks automatiques). Lea ne peut plus faire par erreur :

```
git push origin main
```

si la protection est bien configuree. Elle devra pousser sa branche et ouvrir une PR. Sur GitHub, dans le depot : Settings, puis la zone Branch protection / Rules. Tu choisis la branche `main`. Tu actives des options du genre : exiger une pull request avant merge, exiger un certain nombre d'approbations, exiger que les status checks passent (CI), interdire le force push, interdire la suppression de la branche.

★ A RETENIR

La regle ecrite dans le README est fragile. La regle technique sur `main` tient meme un vendredi a 19h.

Ce que ca change au quotidien

Max finit un fix. Il pousse `fix/login`. Il ouvre une PR vers `main`. Lea approuve. Max clique Merge. `main` avance. Le depot a refuse les raccourcis. Si quelqu'un tente un push direct, le serveur dit non. C'est sec. C'est bon. Pour une equipe de trois, un bon minimum est : pas de push direct, PR obligatoire, au moins une review, pas de force push sur `main`. Les ecrans changent selon les versions de GitHub. L'intention reste : regles sur `main`, filet visible.

Si tu exiges que les checks soient verts avant merge, une **CI** legere (chapitre 8) devient un garde-fou automatique. Review humaine plus tests automatiques egal double filet. Ni l'un ni l'autre n'est parfait seul. Parfois les admins peuvent bypasser. C'est tentant. Utilisez cette exception rarement, et seulement pour des urgences vraiment discutees. Si les admins bypassent tous les jours, vous n'avez plus de protection : vous avez un theatre.

⚠ ATTENTION

Si les admins bypassent la protection tous les jours, vous n'avez plus de filet : vous avez un theatre. Reserve le bypass aux vraies urgences discutees.

Avant : Max pousse sur `main` un hotfix a 19h12. Le site casse. Lea tire `main` le lendemain et passe une heure a comprendre. Personne n'a review. Aucune CI. Apres : Max pousse `fix/...`, ouvre une PR, Sam regarde deux minutes, CI verte, merge. Meme urgence relative, mais un filet. Si la CI est rouge, Max corrige avant d'infecter `main`. A trois personnes, une approval suffit souvent. A dix, parfois deux. N'impose pas trois approvals si vous etes trois et que deux sont en conges : vous vous bloquez.

Sam aime dire : "le serveur dit non, ce n'est pas moi qui suis mechant". Ca desarme les egos. Lea sourit. Max pousse sa branche. Le flux tient.

Petite histoire

L'equipe a active la protection sans prevenir personne. Tout le monde a panique : "Git est casse." Sam a explique en cinq minutes. Deux semaines plus tard, personne ne voudrait revenir en arriere. "On est que deux, pas besoin." Justement : a deux, un clic malheureux n'a pas de troisieme personne pour rattraper dans la seconde. La protection est rapide a activer et evite des soirees tristes.

Chez DanielCraft, on active souvent la protection le jour 1 du depot de test, avant meme le premier vrai feature. Habitude avant vitesse.

Erreur classique

Activer vingt regles d'un coup (3 reviews, 12 checks, signatures) au point de bloquer toute livraison. Commence simple. Explique. Ajuste. Autre piege : proteger `main` mais laisser `master` ou une vieille branche de prod non protegee. Verifie quelle branche est vraiment deployee. On peut proteger les tags plus tard (chapitre 9). Pour commencer, protege `main`.

♦ ASTUCE

Minimum utile a trois : PR obligatoire, une review, pas de force push sur `main`. Tu pourras durcir ensuite.

En vrai

Sur un depot de test (pas le client critique), active une regle : PR obligatoire sur `main`. Demande a un collegue (ou a ton second compte) d'essayer un push direct. Voyez le refus. Puis faites le chemin PR. Le corps retient mieux qu'un paragraphe. Si tu es solo, cree une seconde branche et tente le push : le message d'erreur suffit a ancrer.

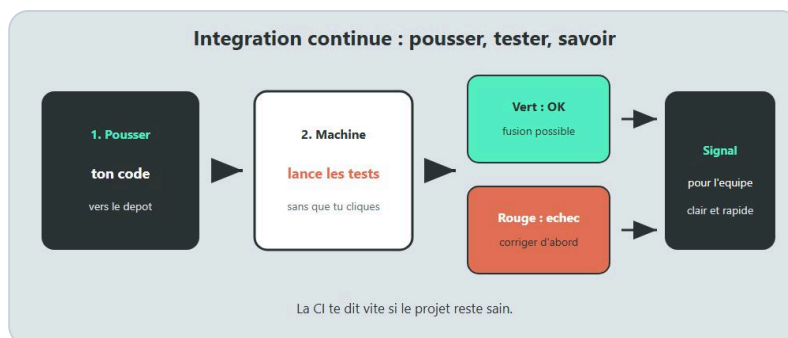
A toi

Note dans le README : "main est protegee : pas de push direct, PR plus 1 review minimum." Ajoute le lien vers les settings si utile. La regle ecrite plus la regle technique egal coherence. Bonus : ajoute une phrase sur le bypass admin ("urgence discutee seulement").

Zoom : filet visible, egos calmes

Sam aime le moment où le serveur dit non. Ce n'est plus "Max est méchant". C'est "la branche est protégée". Lea sourit. Max pousse sa feature. Chez DanielCraft, on active souvent la protection le jour 1 du dépôt de test, avant même le premier vrai feature. Habitude avant vitesse. Tu peux durcir ensuite. Tu peux rarement assouplir sans drame si la prod a déjà brûlé.

Chapitre 8 - CI legere : des tests automatiques sur la PR



La CI verifie la PR avant de fusionner.

La revue humaine regarde le sens. La **CI** regarde la mecanique : est-ce que ca build ? Est-ce que les tests passent ? Est-ce que le linter crie ? Tu n'as pas besoin d'une usine NASA. Tu as besoin d'un feu vert / feu rouge simple sur chaque pull request. CI veut dire Continuous Integration : a chaque proposition de changement, une machine rejoue des verifications. Chez DanielCraft, on aime la CI qui tient sur une page de config et qui finit vite. Si la CI met quarante minutes, les gens la contournent mentalement.

Lea ouvre une PR. GitHub Actions (ou GitLab CI, ou autre) lance un petit script : installer, tester, parfois builder. Au bout de deux minutes, un point vert ou rouge apparait sur la PR. Max voit le rouge avant de merger. Lea corrige. On gagne une categorie entiere de "ouille, j'avais oublie". Imagine le site de Lea, Max et Sam. Ils ont quelques **tests** automatiques sur des fonctions de validation (email, telephone). Sur chaque PR, la CI fait : recuperer le code de la PR, installer les dependances, lancer les tests, afficher succes ou echec. Pas besoin de deployer en production depuis la CI le premier jour. Juste verifier.

★ A RETENIR

Une CI legere et lisible bat une mega config que personne ne comprend. Vert ou rouge, vite, clair.

L'idée sans jargon

Sur GitHub Actions, un fichier dans `.github/workflows/ci.yml` decrit le travail. L'idée generale, pas un tutoriel exhaustif :

```

on: pull_request
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - recuperer le code
      - installer Node (ou Python, etc.)
      - npm test (ou pytest, etc.)
  
```

Tu adapteras a ton stack. L'important est le reflexe : PR egal verification automatique. Quand c'est rouge, le message doit aider. "3 tests failed" avec le nom du test bat "Error". L'auteur corrige sans honte. La CI n'est pas la pour punir. Elle est la pour attraper tot. Si la CI est **flaky** (rouge au hasard), corrige-la vite. Une CI menteuse est pire que pas de CI : on ignore le rouge.

⚠ ATTENTION

Une CI flaky (rouge au hasard) est pire que pas de CI : l'equipe apprend a ignorer le rouge. Repare-la ou retire le check.

Tu peux exiger que le check `test` soit vert avant merge. Alors plus personne ne merge "on verra bien". Le bouton Merge attend le vert. Pour commencer : installer plus tests unitaires rapides. Ensuite eventuellement lint. Ensuite eventuellement un build. Evite d'ajouter dix outils le meme jour. Chaque outil doit gagner sa place. Si vous n'avez aucun test encore, une CI qui lance au moins un script `npm test` (meme minimal) cree le rituel. Puis vous ajoutez de vrais tests au fil des bugs. La CI ne remplace pas la revue humaine sur le sens produit. Elle ne garantit pas qu'un bouton est joli. Elle attrape les regressions mecaniques. Garde les deux filets.

Parfois la CI a besoin d'un token. Stocke-le dans les `secrets` du depot (reglages GitHub), pas dans le fichier yaml. Le chapitre 18 insiste : les cle ne vivent pas dans Git. Semaine 1 : tests unitaires. Semaine 3 : lint. Plus tard : build de preview. Chaque ajout doit reduire une douleur reelle. Si personne ne regarde un check, retire-le ou repare-le. Tout le monde possede la CI. Si c'est "le truc de Sam", Sam en vacances egal CI rouge ignoree.

Petite histoire

Max a copie une mega config internet avec cache, matrix, deploy, notifications Slack. Personne ne comprenait pourquoi c'etait rouge. L'equipe a tout retire et remis un seul job `npm test`. Deux minutes, vert ou rouge lisible. Moral remonte. Investis cinq minutes pour rendre le rouge lisible. Toute l'equipe gagne a chaque PR.

Lea a casse volontairement un test sur une PR de demo. Elle a vu le rouge. Elle a corrige. Elle a vu le vert. Ce petit theatre a ancre le geste mieux qu'un powerpoint. Chez DanielCraft, on adore ce theatre.

Erreur classique

Copier une mega config internet et ne plus comprendre pourquoi c'est rouge. Ou bloquer le merge sur un check optionnel qui casse souvent. Ou n'avoir de CI que sur `main` apres merge : trop tard, le mal est deja integre. Ou croire que la CI remplace toute review humaine.

♦ ASTUCE

Commence par un seul job rapide sur `pull_request`. Ajoute lint et build seulement quand ils gagnent une vraie douleur.

En vrai

Ajoute un test minuscule qui echoue si une fonction `validerEmail` accepte `"a@b"`. Ouvre une PR qui casse volontairement. Regarde le rouge. Corrige. Regarde le vert. Ce petit theatre ancre le geste. Si tu n'as pas encore GitHub Actions, ecris d'abord la phrase README : le rituel commence par l'intention ecrite.

A toi

Ecris la promesse de votre CI en une phrase dans le README : "Chaque PR lance les tests ; on ne merge pas au rouge." Si vous n'avez pas encore de CI, écris la phrase et mets la mise en place dans le mini-projet (chapitre 12) ou juste après. Bonus : chronomètre un run vert. S'il dépasse cinq minutes sans raison, simplifie.

Zoom : vert lisible, rouge utile

Une CI qui dit "Error" sans nom de test, c'est un feu rouge opaque. Investis cinq minutes pour rendre le message lisible. Lea l'a fait après une après-midi perdue. Max aussi. Sam refuse les configs copiées sans lecture. Chez DanielCraft, la CI légère et lisible bat la mega usine ignorée.

Chapitre 9 - Tags et releases



v1.2.0 : un point fixe livre.

Un commit a un hash du genre `a3f19c2`. Pratique pour Git, penible pour les humains. Un **tag**, c'est un nom stable colle sur un commit : `v1.0.0`, `v1.1.0`, `v2.0.0`. Une **release**, c'est souvent ce tag plus des notes ("ce qui change pour les utilisateurs"). Quand Sam dit "le bug est sur la 1.2", tout le monde doit pouvoir retrouver le meme code. Les tags servent a ca. Chez DanielCraft, une release courte et honnete bat un roman marketing.

Tu livres le site. Ca marche. Tu marques le commit actuel :

```
git tag -a v1.0.0 -m "Premiere version publique du site"
git push origin v1.0.0
```

`-a` cree un tag annote (avec message et auteur). C'est en general preferable aux tags legeres pour les versions. Plus tard, tu pourras revenir a ce point precis, construire a partir de lui, ou comparer :

```
git checkout v1.0.0
```

(tu seras en "detached HEAD" : normal pour inspecter une version ; reviens sur une branche apres)

```
git switch main
```

Beaucoup d'equipes utilisent Major.Minor.Patch : `v1.2.3`. Patch : correctif sans changer l'usage (`v1.2.3` -> `v1.2.4`). Minor : nouveaute compatible (`v1.2.4` -> `v1.3.0`). Major : changement qui casse des usages (`v1.3.0` -> `v2.0.0`). Pour un site vitrine, tu peux simplifier : `v1`, `v2`, ou des dates. L'important est d'etre coherent dans l'equipe.

★ A RETENIR

Un tag publie est un point fixe. `main` avance ; `v1.2.0` reste le souvenir d'un instant.

Release sur GitHub

Sur GitHub, tu peux créer une Release à partir d'un tag : titre, notes, fichiers attachés parfois. Lea écrit : "Ajout page tarifs, fix formulaire email". Max déploie en sachant quoi tester. Le client comprend ce qui a bougé. Souvent la personne qui livre, après le merge sur `main`, sur le commit de `main` à jour. Pas sur une feature random. Pas "je tag chez moi sans pousser" : les autres ne verront rien.

```
git switch main
git pull
git tag -a v1.2.0 -m "Tarifs + fix email"
git push origin v1.2.0
```

Une branche bouge. Un tag, en principe, non. `main` avance. `v1.2.0` reste le souvenir d'un instant. C'est pour ça que c'est utile : point fixe dans un fleuve. Évite de déplacer un tag déjà publié (`git tag -f` puis force push) sauf correction immédiate et annoncée. Les gens ont peut-être déjà tiré `v1.2.0`. Idéalement : le même commit tagué est celui que tu déploies. Si tu déploies un commit et tu tags un autre, tu mens à ton futur toi. Aligne les gestes : merge, tag, deploy, notes.

⚠ ATTENTION

Déplacer un tag déjà publié (`git tag -f`) sans annonce, c'est mentir à ceux qui l'ont déjà tiré. Ne le fais que pour une correction immédiate et dite à voix haute.

Mauvais : "divers fixes". Meilleur : "Fix validation email du formulaire contact. Ajout section tarifs sur /offres. Pas de migration base." Écris pour la personne qui va tester et pour celle qui va communiquer au client. Un bug grave sur `v1.3.0`. Tu sais que `v1.2.0` était sain. Tu peux inspecter, comparer, ou redéployer l'ancien artefact issu de ce tag selon votre pipeline. Sans tag, tu fouilles les dates et les souvenirs. Avec tag, tu as un nom :

```
git log --oneline v1.2.0..v1.3.0
```

Tu vois ce qui a changé entre les deux. Utile pour enquêter (complément de bisect parfois). Les tags légers sont juste un nom. Les **annotés** portent un message et des métadonnées. Pour les versions, préfère annoter.

Sam dit "on est sur la 1.2". Lea ouvre le tag. Max déploie le même commit. Trois cerveaux, un même point fixe. C'est toute la promesse.

Petite histoire

Lea a tagué `v1` dix fois en le déplaçant. Max a déployé une version, Sam en a testé une autre. Personne ne savait quelle était "la 1.2". L'équipe a décidé : un tag publié ne bouge plus. Notes courtes. Tag pousse. Déploiement aligné. Vendredis plus calmes.

Chez DanielCraft, on préfère trois lignes de notes honnêtes à une page marketing. Le client et le futur toi te remercieront.

Erreur classique

Ne jamais taguer, puis chercher pendant une heure "la version de mardi". Ou mettre dans les notes de release des secrets (non). Ou oublier de pousser le tag : tout le monde croit que la 1.2 n'existe pas. Ou taguer depuis une branche feature au lieu de `main` a jour.

♦ ASTUCE

Après un merge utile sur `main` : pull, tag annotate, push du tag, notes courtes. Aligner tag et déploiement.

En vrai

Sur le dépôt de test, crée `v0.1.0` sur `main`, pousse le tag, crée une Release GitHub avec trois lignes de notes. Ensuite avance `main` d'un commit et crée `v0.1.1`. Liste :

```
git tag
```

Sens la chronologie. Relie mentalement chaque tag à "ce qu'on a livré". Si tu ne peux pas le dire en une phrase, les notes sont trop floues.

A toi

Décide avec l'équipe le schéma de versions (SemVer simple ou autre) et qui a le droit de publier une release. Une phrase dans le README suffit : "On tague main après chaque livraison utile : vX.Y.Z." Bonus : ajoute "notes courtes, pas de secrets".

Zoom : nommer pour retrouver

Sans tag, "la version de mardi" devient une chasse. Avec tag, tu pointes. Lea tague après chaque livraison client. Max tague après chaque hotfix hot. Sam enseigne SemVer en cinq minutes : majeur / mineur / patch. Chez DanielCraft, on préfère un schéma simple tenu à un schéma savant abandonné.

Chapitre 10 - Cherry-pick : reprendre un commit ailleurs



Ramener un commit utile ailleurs.

Parfois tu as un commit parfait... mais pas sur la bonne branche. Le fix est sur une feature longue. La prod brule. Tu voudrais juste cette correction sur `main`, sans merger toute la feature. `git cherry-pick` copie un commit (son **diff**) et le rejoue sur ta branche actuelle. Nouveau **hash**, meme idee. Chez DanielCraft, on dit : tu cueilles une **cerise** sur un arbre (une branche) et tu la poses sur un autre plateau. Tu ne demenages pas l'arbre. Tu ne contournes pas la review non plus : tu ouvres quand meme une PR courte.

Max a trois commits sur `feature/refonte-login` :

```
a111 - prepare la structure
b222 - corrige le timeout session <-- celui-la est urgent
c333 - nouveau design du formulaire
```

La prod a besoin de `b222` maintenant. Merger toute la feature amenerait le design inacheve. **Cherry-pick** sur une branche `fix/` courte, puis **PR** express :

```
git switch main
git pull
git switch -c fix/timeout-session
git cherry-pick b222
```

Si ca s'applique proprement, tu as le fix sur une branche courte. Tu ouvres une PR. Tu merges. La feature de Max continue sa vie. Plus tard, quand la feature rejoindra `main`, Git gerera souvent le doublon assez bien (pas toujours magique : reste attentif). Trouve le commit :

```
git log --oneline feature/refonte-login
```

Tu copies le hash court ou long. Tu cherry-pick ce hash.

★ A RETENIR

Cherry-pick egal une cerise urgente, pas le panier entier. Pour une feature complete, passe par une PR normale.

Conflits et limites

Comme un merge ou un rebase, ca peut conflictuer. Tu resolu, tu `git add`, tu continues :

```
git cherry-pick --continue
```

Ou tu abandonnes :

```
git cherry-pick --abort
```

Tu peux enchaîner plusieurs cherry-picks, ou donner un intervalle selon les cas. Pour debuter, un commit a la fois. C'est plus clair mentalement. Cherry-pick brille pour 1 (parfois 2-3) commits cibles. Pour une feature entiere : merge ou PR normale. Si tu cherry-picks vingt commits d'une feature, tu es en train de merger a la main, mal. Apres cherry-pick, tu as deux commits cousins (meme changement, hash differents) sur deux lignes d'histoire. Les messages restent importants.

⚠ ATTENTION

Si le commit cueilli depend d'un commit precedent non pris, tu peux casser la compilation. Lis `git show` avant de cueillir.

Sam : "Le timeout session casse la prod." Max : "Le fix est dans ma grosse branche refonte, commit b222." Sam : "On cherry-pick b222 sur une fix branche, PR express, on livre. Ta refonte continue." C'est ca, l'outil au service du calendrier. Pas un tour de magie pour eviter le travail : une extraction ciblee. Si `b222` suppose que `a111` a cree une fonction, cherry-pick de `b222` seul peut casser la compilation. Alors tu cherry-picks `a111` puis `b222`, ou tu recrees un fix minimal sur `main` a la main. Lis le commit avant de cueillir la cerise :

```
git show b222
```

Lea a appris a noter dans la PR feature : "timeout deja livre via cherry-pick le ...". Quand la grosse branche arrive, moins de surprise.

Petite histoire

Lea a cherry-picke depuis le mauvais hash. Le code ne compilait plus. Elle a lu `git show`, recommence proprement, PR courte, merge. Quand la refonte de Max est arrivee, Git a vu des changements deja presents. Un conflit "deja applique". Note dans la PR feature : "timeout deja livre via cherry-pick le ...". Respire, resols, continue.

Chez DanielCraft, cherry-pick n'est pas un plan B pour eviter la review. Tu ouvres quand meme une PR courte. Le filet reste.

Erreur classique

Cherry-pick depuis le mauvais hash. Ou cherry-pick d'un commit qui depend d'un commit precedent non pris. Ou cherry-pick directement sur `main` sans PR alors que `main` est protegee... passe par une branche fix. Autre piege : utiliser cherry-pick pour "eviter la review". Non. Tu ouvres quand meme une PR courte.

♦ ASTUCE

Toujours `git show <hash>` avant `git cherry-pick <hash>`. Une minute de lecture evite une heure de debug.

En vrai

Sur un depot de test, cree une branche avec deux commits distincts (deux fichiers differents). Reviens sur `main`, cherry-pick seulement le second. Verifie que seul le second changement est la :

```
git log --oneline -5
git show HEAD
```

Puis ouvre une PR courte meme sur le depot de test. Le geste compte autant que la commande.

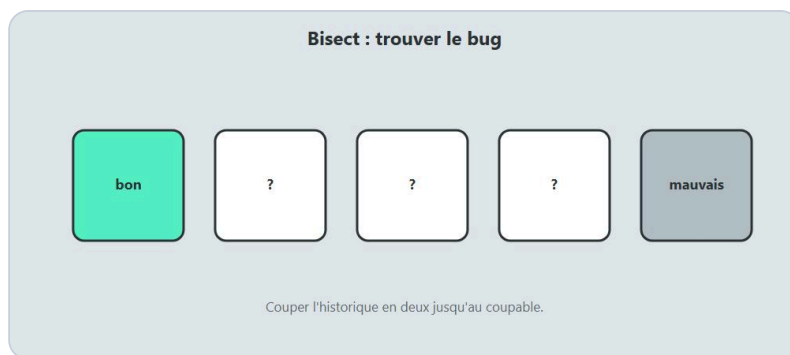
A toi

Ecris dans ton carnet d'equipe : "Cherry-pick egal cerise urgente, pas panier entier." Quand la prod brulera, tu auras la phrase au lieu de la panique. Bonus : ajoute "toujours `git show` avant de cueillir".

Zoom : urgence sans chaos

La prod brule. La feature est longue. La tentation, c'est de merger tout ou de pousser direct. Cherry-pick + branche fix + PR courte, c'est le chemin du milieu. Lea l'a appris apres un vendredi trop long. Max aussi. Sam l'enseigne avant l'incendie. Chez DanielCraft, on prefere une cerise bien posee a un panier renverse sur `main`.

Chapitre 11 - Bisect : trouver le commit qui a casse



Couper l'historique pour trouver le bug.

Il y a deux semaines, le formulaire marchait. Aujourd'hui, non. Entre les deux, vingt commits. Personne ne sait lequel a casse. Lire chaque diff a la main est long. `git bisect` fait une recherche par dichotomie : il te fait tester le milieu, puis la moitié restante, jusqu'au premier commit coupable. Chez DanielCraft, **bisect** est le detecteur gentil de l'historique : pas pour blamer une personne, pour trouver un changement.

Tu dis a Git : "la, c'etait bon" (un vieux commit). "la, c'est mauvais" (souvent HEAD). Git te pose au milieu. Tu testes. Tu dis bon ou mauvais. Git réduit. En quelques etapes, tu as le commit qui a introduit le bug. L'outil te guide. Toi, tu restes calme et methodique.

★ A RETENIR

Bisect trouve un changement. Il ne juge pas un collegue. On corrige, on ajoute un test si possible, on avance.

Le deroulement

```
git bisect start
git bisect bad
git bisect good v1.1.0
```

Ici, tu marques la version taguee `v1.1.0` comme bonne, et l'etat actuel comme mauvais. Git checkout un commit du milieu. Tu testes le site (formulaire, page, test auto...).

Si c'est encore casse :

```
git bisect bad
```

Si ca marche :

```
git bisect good
```

Tu repetes. A la fin, Git affiche le premier commit mauvais. Tu lis le diff :

```
git show
```

Puis tu sors du mode bisect :

```
git bisect reset
```

Tu reviens ou tu etais. Ensuite tu corriges (souvent une nouvelle branche fix depuis `main`).

Automatiser un peu

Si tu as un test qui echoue de facon fiable, tu peux laisser Git lancer le test a ta place (`git bisect run ...`). Utile, un cran plus avance. Pour commencer, le mode manuel suffit : tu ouvres le navigateur, tu cliques, tu dis bon/mauvais. Lea prefere le manuel au debut : elle voit le site, elle comprend. Max automatise des qu'il a un test solide.

Bien choisir good et bad

Il te faut un point vraiment bon. Un `tag` de release aide. Ou un commit que tu te souviens avoir teste. Si ton "good" n'est pas vraiment bon, bisect te menera nulle part utile.

Bad = l'etat casse. Good = l'etat sain. Ne les inverse pas : tu obtiendrais le message inverse de la realite. Note ta procedure de test sur un papier (trois clics). Reproduis toujours pareil. Si tu changes de methode au milieu, tu pollues la recherche.

⚠ ATTENTION

Oublier `git bisect reset` laisse le depot dans un etat etrange. Des que tu as le coupable, reset, puis branche fix.

Mardi, le bouton Envoyer ne fait plus rien. Vendredi dernier, demo client OK. Entre les deux, dix-huit commits. Sam lance bisect avec le tag `v0.2.0` comme good. Huit tests manuels plus tard, Git pointe un commit "ajoute tracker clic" qui avait casse le handler. Pas de chasse aux sorcieres : un revert cible ou un fix, plus un test. L'apres-midi est sauvee.

Ferme les caches navigateur si besoin. Tu testes le code d'un commit ancien : normal que le site ait l'air "en retard". Tu n'enquetes pas sur le design. Tu enquetes sur le bug.

Ce que bisect n'est pas

Ce n'est pas une excuse pour humilier l'auteur du commit. Le commit "coupable" est une info technique. Peut-etre que le bug dependait d'un environnement. Peut-etre que le test manquait. On corrige. On ajoute un test si possible. On avance. Ce n'est pas non plus magique si le bug est `flaky` (parfois oui, parfois non) : bisect souffre. Si le bug vient d'une donnee externe, pas du code, bisect ne peut pas inventer la cause. Si plusieurs commits combines causent le probleme, tu trouveras souvent le premier qui fait basculer le test : lis le contexte autour.

Petite histoire

Lea cherchait au hasard pendant une heure. Sam a dit : "tu as un tag bon ?" Oui, `v0.2.0`. Bisect. Huit étapes. Commit trouve. Max a ajoute un test pour que la CI attrape ca la prochaine fois. Trois roles, une meme lecon : methode avant panique.

Chez DanielCraft, on affiche parfois la phrase au-dessus de l'ecran : "Avant de chercher au hasard, tente bisect."

Erreur classique

Oublier `git bisect reset` et rester dans un etat etrange. Ou marquer bad/good a l'envers. Ou chercher au hasard pendant une heure alors qu'un tag `v1.1.0` "bon" etait sous la main. Autre piege : changer la procedure de test au milieu de la recherche.

♦ ASTUCE

Note trois clics de test sur un papier avant de lancer bisect. Reproduis toujours la meme procedure.

En vrai

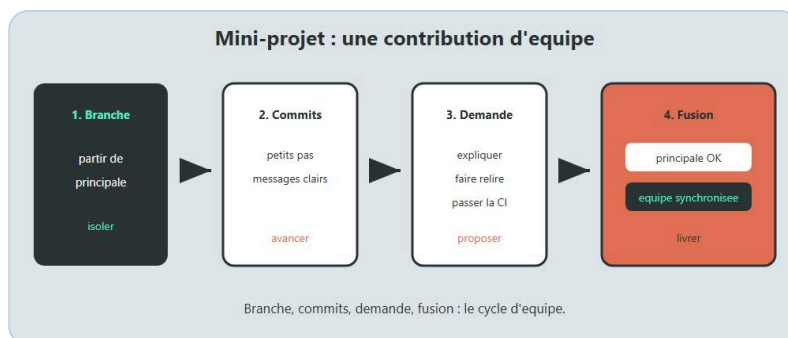
Dans un depot de test, introduis volontairement un bug a un commit (par exemple une fonction `addition(a, b)` qui retourne `a - b`), puis ajoute d'autres commits innocents. Lance bisect depuis un bon tag jusqu'a HEAD. Verifie que Git trouve le commit faute. C'est le meilleur tutoriel.

Apres avoir trouve : `git bisect reset`, puis branche fix depuis `main`, correctif, test, PR. Eventuellement ajoute un test automatique pour que la CI attrape ca la prochaine fois. Bisect t'a montre le ou ; la CI evite d'y retourner.

A toi

Ajoute dans tes notes : "Avant de chercher au hasard pendant une heure, je tente bisect si j'ai un point good." Le reflexe compte plus que la syntaxe parfaite.

Chapitre 12 - Mini-projet : simulation d'equipe



Mini-projet : simuler une vraie collab.

Assez de theorie separee. On colle les briques. Tu vas simuler une petite equipe sur un mini site : page d'accueil, page offres, formulaire de contact. Lea, Max, Sam (meme si tu joues les trois roles tout seul). But : vivre le **flux** complet une fois, proprement. Chez DanielCraft, un mini-projet fait avec les mains bat dix chapitres lus sans depot.

Vous livrez **v0.1.0** d'un site vitrine. Puis Lea ajoute une section tarifs sur la page offres. Max corrige un bug : le formulaire accepte un email invalide. Sam protege **main**, ajoute une CI minimale si possible, et publie **v0.2.0**. Tu peux tout faire sur un seul compte en changeant de branches et en ouvrant des PR. Idealement, un ami joue le reviewer.

★ A RETENIR

L'objet du mini-projet, c'est le flux, pas le pixel perfect. Un bandeau moche versionne proprement bat une refonte poussee n'importe comment sur main.

Preparation

Cree un depot GitHub vide (ou local + remote). Clone. Sur **main**, pose un site minimal : **index.html**, **offres.html**, **contact.html**, un peu de CSS. Commit initial clair. Pousse **main**.

```
git switch -c main
# si besoin : premier commit deja sur main
git push -u origin main
```

Active une **protection** simple sur **main** : PR obligatoire (chapitre 7). Sans ce filet, tu rates une partie de l'exercice.

Role Lea : feature tarifs

```
git switch main
git pull
git switch -c feature/section-tarifs
```

Ajoute la section. Commits clairs (contenu, puis style si tu veux). Pousse. Ouvre une PR avec but + comment tester. Fais reviewer (ami ou toi via autre oeil). Merge. Tire **main**.

Role Max : fix email

Pendant ou apres Lea, Max part de `main` a jour :

```
git switch main
git pull
git switch -c fix/validation-email
```

Corrige la validation. Ajoute un test minuscule si tu peux (meme une fonction JS + un runner simple). PR. Review. Merge. Si la `CI` existe, regarde le vert. Si elle n'existe pas encore, Sam s'en occupe ensuite, ou tu l'ajoutes dans une PR `chore/ci-legere`.

Role Sam : filets et release

Sam verifie que `main` est protegee. Sam ajoute une CI legere qui lance les tests sur les PR. Sam, apres les merges utiles :

```
git switch main
git pull
git tag -a v0.2.0 -m "Tarifs + validation email"
git push origin v0.2.0
```

Cree une Release GitHub avec trois lignes de notes. Option bonus : cherry-pick si tu as laisse trainer un fix urgent sur une branche longue. Autre bonus : introduis un bug, retrouve-le avec bisect, corrige, tague `v0.2.1`.

♦ ASTUCE

Chronometre. Une demi-journee suffit souvent. Note ce qui frotte : c'est le vrai cours.

Criteres "c'est reussi"

`main` n'a reçu que des merges via PR. Les messages de commit sur les features sont lisibles. Au moins une review a eu lieu (meme simulee). Un tag `v0.2.0` existe. Tu peux expliquer a voix haute le chemin d'une idee jusqu'a la release. Une personne exterieure peut cloner, lire le README "Comment on travaille", et comprendre comment proposer un changement sans dix questions.

Planning suggere (demi-journee)

0h00-0h30 : socle site + remote + protection main. 0h30-1h30 : role Lea (feature tarifs + PR + merge). 1h30-2h30 : role Max (fix email + test + PR). 2h30-3h15 : role Sam (CI minimale + tag v0.2.0 + notes). 3h15-3h30 : retro ecrite dans le README. Si tu es seul, enchaene les casquettes. Si vous etes trois, prenez les roles pour de vrai et chronometrez les attentes de review : ca enseigne le "repondre vite".

Petite histoire

Lea a voulu peaufiner le design pendant l'atelier. Max lui a dit : "HTML minimal, on versionne le flux." Ils ont livre `v0.2.0` avant 16h. Sam a ecrit la retro : "protection activee trop tard, on l'active des le jour 1 la prochaine fois." Trois lecons dans une demi-journee. Chez DanielCraft, c'est exactement le but.

Erreur classique

Tout faire directement sur `main` "pour aller plus vite" : tu rates l'exercice. Ou ouvrir une seule PR geante Lea+Max+Sam. Decoupe. Le mini-projet entraine le decoupage. Autre piege : construire un design system pendant l'atelier Git.

⚠ ATTENTION

Ne construis pas un design system ici. L'objet, c'est le flux d'equipe. HTML/CSS minimal suffit.

En vrai

Chronometre. Une demi-journee suffit souvent. Note ce qui a frotte : protection, conflits, description de PR. Ces frottements sont le cours. A la fin, une personne exterieure (ami, collegue) peut-elle comprendre votre README sans vous poser dix questions ? Si oui, vous avez reussi plus que "faire des commandes".

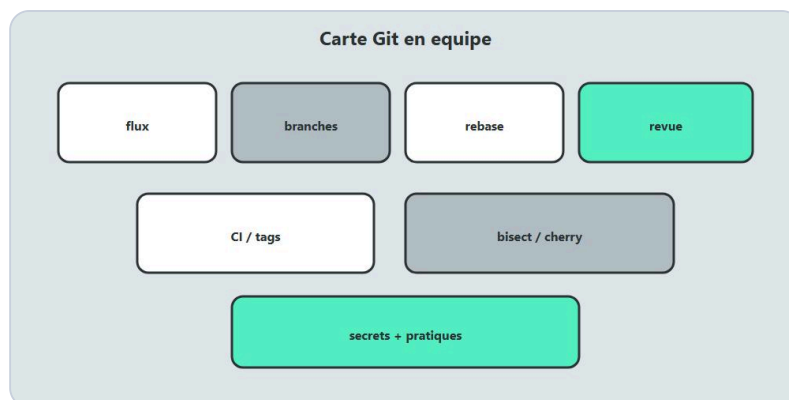
A toi

A la fin, ecris un paragraphe "ce que notre equipe a decide" : flux, noms de branches, rebase ou merge, qui release. Colle-le dans le README. Le mini-projet devient votre contrat leger. Bonus : coche protection `main` + au moins une PR mergee + un tag. Trois coches, un vrai kit.

Zoom : demi-journee, vrai contrat

Le but n'est pas un site joli. Le but, c'est un depot ou le prochain mardi sera plus calme. Lea a voulu peaufiner le design. Max a rappele le flux. Sam a note la retro. Chez DanielCraft, cette demi-journee vaut dix lectures du chapitre flux - a condition d'ecrire le contrat a la fin.

Chapitre 13 - A retenir



Carte Git en equipe.

Tu n'as pas besoin de tout memoriser par coeur. Tu as besoin d'une carte mentale stable. Voici la version poche du livre, a relire avant une journee d'equipe. Chez DanielCraft, Git en equipe, ce n'est pas "connaître plus de commandes". C'est "se synchroniser sans se blesser". Les commandes servent ce but. Si tu peux dire ça a voix haute, tu as la colonne vertebrale.

Imagine un tableau blanc au milieu de la piece. `main` distant, c'est le tableau officiel. Lea, Max et Sam ont chacun une copie. Sans `flux`, chacun dessine et s'ecrase. Avec `flux` : tire le tableau, dessine sur ton coin (`branche`), propose (`PR`), regarde, colle, nettoie. Petit et frequent bat grand et rare. Relis cette image avant chaque session confuse. Elle vaut mieux qu'un catalogue de sous-commandes.

★ A RETENIR

Se synchroniser sans se blesser. Tout le reste du livre sert cette phrase.

Le flux et les branches

Tire `main` en debut de session. Cree une branche pour une intention. Commit. Pousse la branche. Ouvre une PR. Fais reviewer. Merge. Retire `main`. Recommence. `main` = reference. Feature/fix = travail. Noms clairs. Duree courte. Une intention par branche. Nettoie apres merge. GitHub Flow leger suffit a 2-5 personnes. Lea tient des features courtes. Max aussi. Sam coupe les branches zombies. Si une branche vit trop longtemps, elle devient un conflit annonce.

Le rituel du matin tient en trois commandes et une intention. `git switch main`, `git pull`, puis une branche nommee pour la tache du jour. Le soir, si la feature avance, pousse. Si elle est prete, ouvre la PR avec but et comment tester. Apres merge, tire `main` et efface la branche morte. Ce cycle, repete, bat n'importe quelle usine a process copiee d'une grosse boite.

Rebase, merge, historique

`Merge` joint les histoires. `Rebase` rejoue tes commits sur une base plus recente. Utile sur une feature perso. Dangereux sur l'histoire partagee si tu force-push sans soin. Pas de dogme : une politique d'equipe ecrite. Messages qui disent le pourquoi. Petits commits coherents. Squash possible a l'integration. Amend seulement en local / branche perso. Pas de secrets dans Git. Chez DanielCraft, on ecrit la politique en dix lignes dans le README. Pas dans un wiki perdu.

Si tu te demandes "merge ou rebase ?", pose d'abord : "cette branche est-elle partagee ?" Si oui, merge est souvent plus doux. Si non, rebase sur `main` avant la PR peut clarifier le log. Dans les deux cas, un conflit demande un choix humain. `git rebase --abort` ou `git merge --abort` existent pour respirer.

Revue, protection, CI

Bienveillante, utile, rapide. Description claire cote auteur. Regard priorise cote reviewer. Dire aussi ce qui est bien. Pas d'Approve fantome. Pas de tribunal. `main` protegee : pas de push direct, PR, review. CI legere : tests sur la PR, feu vert avant merge. Filet technique + filet humain. L'un sans l'autre laisse un trou. Lea aime le filet humain. Max aime le filet technique. Sam exige les deux.

Trois phrases a coller sous l'ecran : "On review pour aider.", "On ne merge pas au rouge.", "On ne pousse pas sur main." Si ton equipe ne retient que ca, elle a deja gagne une categorie de vendredis.

Tags, cherry-pick, bisect, secrets

Nommer un commit livre : `vX.Y.Z`. Pousser le tag. Notes courtes. Aligner tag et deploiement. Cherry-pick : reprendre une cerise ailleurs, surtout en urgence. Bisect : trouver le commit qui a casse. Outils de precision, pas de panique. Fork + upstream pour contribuer a l'exterieur. Jamais de cles dans le depot. Les bonnes pratiques tiennent en quelques phrases ecrites. Relis-les avant de paniquer.

Quand la prod brule, resiste a deux reflexes : pousser direct sur `main`, et merger toute une feature longue. Prefere une branche `fix/` courte, une PR express, un cherry-pick si la cerise est isolee. Quand tu cherches un bug dans vingt commits, tente bisect avant une heure de lecture au hasard. Quand une cle fuit, invalide d'abord, nettoie ensuite.

♦ ASTUCE

Si tu te perds, reviens au chapitre 2 (flux) et au chapitre 6 (revue). Presque tous les frottements viennent de la.

Petite histoire

Lea a voulu tout retenir d'un coup. Elle a bloque. Elle a affiche cette carte au-dessus de l'ecran. Max a saute la revue "parce que c'etait petit" : prod cassee un vendredi. Sam a force-push sur une branche partagee : deux heures de reparation. Trois lecons, une carte. Si tu te perds, reviens au chapitre 2 (flux) et au chapitre 6 (revue). Presque tous les frottements viennent de la.

Chez DanielCraft, on dit souvent : la carte poche vaut mieux que la memoire heroique. Relis. Agis. Ajuste. Avant une journee confuse, dix minutes sur cette page battent une heure de recherche au hasard.

Erreur classique

Croire que "je connais Git" = "je sais collaborer". Les bases sont le moteur. L'equipe, c'est le code de la route. Autre piege : empiler les outils (hooks, bots, templates) avant d'avoir un flux tenu a trois personnes. Autre piege : relire ce recap sans ouvrir le depot. La carte sans volant, ca ne conduit pas.

⚠ ATTENTION

Empiler bots et hooks avant un flux tenu, c'est decorer une maison sans fondations. Pose le flux d'abord.

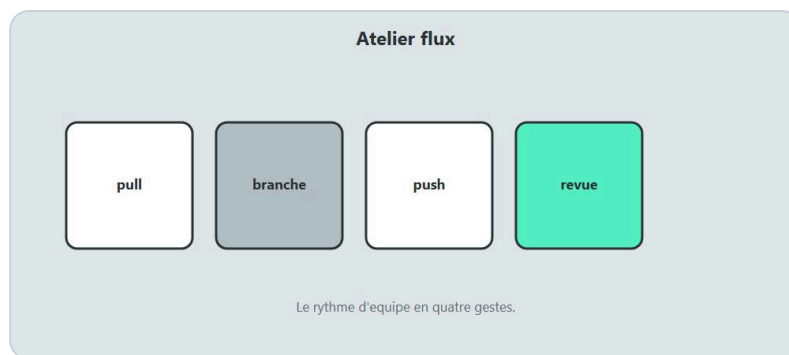
En vrai

Sans regarder le livre, écris sur papier les 8 étapes du flux (pull main ... pull main). Chronometre cinq minutes. Si tu bloques, relis le chapitre 2. Si tu y arrives, tu as la colonne vertebrale. Puis ouvre ton vrai depot et regarde si ta dernière PR suit encore cette carte. Note un écart. Corrige-le cette semaine, pas "un jour".

A toi

Écris en cinq lignes le contrat d'équipe que tu veux : branches, PR, revue, `main`, secrets. Montre-le à un collègue. Ajustez. Collez-le dans le README. Une page tenue bat un discours inspire. Bonus : ajoute une ligne "comment on gère l'urgence" (hotfix via PR courte, pas push direct). Relis ce chapitre avant le quiz : c'est la carte, pas un examen.

Chapitre 14 - Atelier : flux d'equipe



Pull, branche, push, revue.

Cet atelier fait vivre le chapitre 2 avec les mains. Tu joues deux roles : Alex (feature) et Blake (reviewer). Un seul ordinateur suffit. Chez DanielCraft, on prefere un atelier fait a fond a trois ateliers survoles. Chronometre : 45 a 90 minutes. Ecris le **livrable**. Sans livrable, le cerveau classe ca comme "lu", pas comme "su".

Tu vas marcher le **flux** complet une fois : **main** a jour, **branche**, commits, push, **PR**, **review**, **merge**, retour propre sur **main**. Lea fait ca chaque matin sans y penser. Max aussi. Sam a appris en se brulant. Toi, tu le fais lentement une fois pour que ca devienne un reflexe. Le depot de test, pas le client critique. Les mains d'abord, la vitesse ensuite. Si tu sautes une etape "parce que c'est petit", tu rates exactement le muscle a construire.

★ A RETENIR

Le sentier complet, une fois lentement, vaut mieux que dix lectures du chapitre flux.

But et materiel

Partir de **main** a jour, creer une branche, pousser, ouvrir une PR, faire merger, revenir sur **main** proprement. Materiel : depot GitHub de test, Git configure, navigateur, une page **index.html** deja sur **main** (meme minimale). Pas ton vrai projet client. Si tu n'as pas de depot de test, cree-en un en dix minutes. L'atelier vaut ce detour.

Avant de commencer, écris sur un papier les neuf etapes. Coche au fur et a mesure. Le papier force le rythme. Lea le fait encore parfois quand elle forme un stagiaire. Max prefere une checklist dans le README. Sam chronometre. Choisis ton outil, mais ne compte pas sur ta memoire seule.

Etapes

1. Clone ou ouvre le depot, place-toi sur **main**, tire.

```
git switch main
git pull
```

1. 2. Cree **feature/bandeau-accueil** et ajoute un bandeau texte dans **index.html** ("Bienvenue chez nous").

```
git switch -c feature/bandeau-accueil
```

1. 3. Au moins deux commits : HTML, puis un peu de CSS si tu veux. Messages clairs (pourquoi). Pas "fix" et "update". Dis ce que le bandeau change pour l'humain.

1. 4. Pousse la branche (pas `main`).

```
git push -u origin feature/bandeau-accueil
```

1. 5. Ouvre une PR vers `main`. Titre clair. Description : but + "comment tester" (ouvrir l'accueil, voir le bandeau). Verifie `base: main` avant d'envoyer.

1. 6. En mode Blake, relis le diff. Laisse un commentaire bienveillant (meme mineur). Approuve. Si tu es seul, change de casquette : relis comme si tu ne connaissais pas le commit.

1. 7. Merge (bouton GitHub). Coche suppression de branche distante si proposee.

1. 8. En local : reviens sur `main`, tire, supprime la branche locale.

```
git switch main
git pull
git branch -d feature/bandeau-accueil
```

1. 9. Verifie `git status` propre et `git log --oneline -5` coherent. Le bandeau doit etre visible dans le navigateur sur `main`.

♦ ASTUCE

Variante a deux : Alex fait 1-5, Blake 6-7, Alex 8-9. Parlez a voix haute : "PR prete", "review ok", "c'est merge".

Petite histoire

Alex (toi) a voulu pousser direct sur `main` "parce que c'est petit". Blake a refuse. La PR a force une description. Deux jours plus tard, le bandeau avait un typo : la description "comment tester" a aide a le voir avant merge. Lea raconte souvent cette scene : le "petit truc" sans PR, c'est le piege classique. Max sourit. Sam coche la case Delete branch.

Chez DanielCraft, on chronometre parfois l'atelier. Quarante-cinq minutes suffisent si le depot de test est pret. Quatre-vingt-dix minutes si tu debutes vraiment le remote. L'important n'est pas le chronometre. C'est le sentier complet sans raccourci.

Criteres et blocages

Aucun push direct sur `main`. PR avec description. `main` local a jour apres merge. Branche feature nettoye. Protection refuse un geste : c'est voulu, passe par la PR. Conflit : tire `main` dans ta feature, resols, pousse. Mauvais remote : `git remote -v`. Status sale : range avant de continuer. Tu n'es pas en retard. Tu es en train d'apprendre le vrai rythme.

Si la protection n'est pas activee, active-la maintenant (chapitre 7) et recommence l'etape push sur `main` : le refus du serveur ancre mieux qu'un paragraphe. Si tu n'as pas d'ami reviewer, ecris la revue dans `NOTES-REVIEW.md` comme a un collegue, puis Approuve. L'exercice du regard reste valable.

Erreur classique

Sauter la description de PR. Ou merger sans regarder le diff "parce que c'est toi". Autre piege : laisser la branche locale pourrir apres merge. Autre piege : ouvrir la PR vers la mauvaise base. Verifie `base: main` avant de crier victoire. Autre piege : un seul commit "tout" alors que l'atelier demande au moins deux intentions separees.

⚠ ATTENTION

Merger sans regarder le diff "parce que c'est toi", c'est un Approve fantome deguise. Lis quand meme.

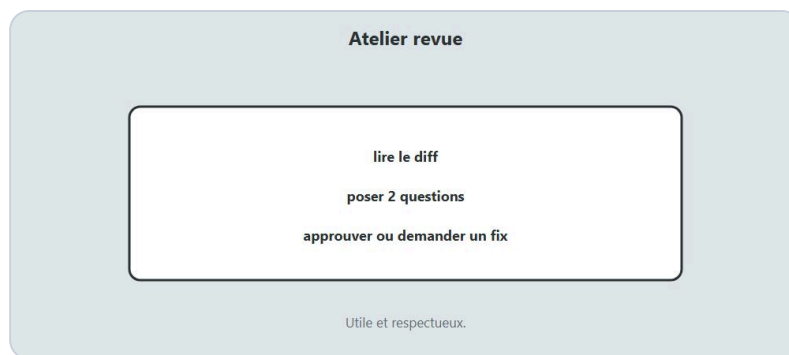
En vrai

Note en trois lignes ce qui t'a freine. Garde la note. Le prochain atelier ira plus vite. Si rien ne t'a freine, note ce qui t'a surpris positivement. Les deux comptent. Relis ta note avant l'atelier revue : tu sauras ou tu tends a raccourcir.

A toi

Fais l'atelier maintenant. Coche les 9 etapes. Ecris le livrable : captures ou notes + trois freins. Range-le dans un dossier "ateliers-git". Dans une semaine, refais juste les etapes 5 a 9 sur une autre micro-feature. Le reflexe s'ancre. Bonus : ajoute dans le README du depot de test la recette en huit lignes du chapitre 2, puis verifie que ton atelier l'a suivie.

Chapitre 15 - Atelier : revue de code



Lire, questionner, approuver.

Ici, le code est un pretexte. L'atelier entraine le regard et le ton. Tu prepareras volontairement une **PR** "presque bien" avec deux vrais points a discuter. Chez DanielCraft, une **revue** bienveillante et utile bat une revue brillante et humiliante. Duree : 45 a 60 minutes. Ecris le **livrable**.

Lea ecrit. Max review. Sam alterne. Toi, tu joues les deux casquettes : **auteur** puis **reviewer**. La **description** aide le reviewer. Le reviewer aide l'auteur. Personne ne "gagne". Le code gagne. Si le ton casse, la prochaine PR sera cachee. Si le ton tient, l'equipe accelere. Tu vas sentir la difference entre "C'est nul" et "Ici, si l'email est vide, on pourrait bloquer l'envoi". Meme fond. Autre monde. C'est exactement le muscle de ce chapitre.

★ A RETENIR

Le ton compte autant que le fond. Une remarque juste dite mal casse la confiance.

But et preparation

Ecrire une PR aidante, puis faire une revue bienveillante et utile. Sur un depot de test, depuis `main` a jour, cree `feature/formulaire-contact-atelier`. Ajoute un formulaire : champ email, bouton envoyer, validation imparfaite (accepte `a@b` sans domaine - voulu), commentaire HTML vague `<!-- temp -->`. Commit en deux temps. Pousse. Ouvre la PR avec vraie description (but, changements, comment tester). N'avoue pas tous les defaults : laisse le reviewer travailler. Si tu avoues tout, tu rates l'entrainement du regard.

Avant d'ouvrir la PR, relis le chapitre 6 en diagonale : priorites, ton, "comment tester". Puis ecris la description comme si Max allait la lire a 18h, fatigue. Trois sections. Pas un roman. Pas "fix". Lea dit que la description est le cadeau. Max a longtemps ecrit "voir le diff". Depuis qu'il detaille, ses reviews durent dix minutes au lieu de quarante.

Etapes auteur

1. Titre : verbe + objet ("Ameliore le formulaire de contact").
2. Remplis but / changements / comment tester.
3. Capture si tu peux.
4. Demande review a un ami, ou change de casquette.

Prends le temps sur "comment tester". Trois lignes claires. Ouvre la page. Tape un email foireux. Clique Envoyer. Note ce que tu attends. Si tu ne peux pas écrire ces trois lignes, la PR n'est pas prête pour un humain extérieur.

Étapes reviewer

1. Lis la description avant le diff.
2. Suis "comment tester" pour de vrai.
3. Au moins deux commentaires utiles : validation email, commentaire `temp`.
4. Formule en suggestions, sans insultes, sans "évident".
5. Dis une chose positive.
6. Request changes si le bug email est bloquant, sinon Comment + discussion.

♦ ASTUCE

Dis une chose positive avant ou après la suggestion. Trois secondes, beaucoup d'énergie pour l'auteur.

Modèles de formulation. Suggestion : "Ici, si l'email est `a@b`, l'envoi part quand même - on pourrait exiger un domaine avec un point." Bloqueur : "La validation laisse passer `a@b` : Request changes pour ce point avant merge." Positif : "Claire la structure du formulaire, facile à tester." Copie ces trois tons. Adapte. Garde-les.

Étapes retour auteur

1. Réponds sans te défendre pour la forme.
2. Corrige la validation. Enlève `<!-- temp -->`.
3. Pousse. "C'est à jour, tu peux revoir".
4. Approuver et merger si c'est bon.

Le "sans te défendre pour la forme" est dur la première fois. Sam le rappelle : tu défends le produit, pas ton ego. Lea respire avant de répondre. Max aussi, depuis une réponse trop sèche un mardi. Si tu n'es pas d'accord sur un point de style, dis-le calmement. Si le bug email est réel, corrige sans débat d'ego.

Petite histoire

Lea a Approuve sans tester : bug en prod. Max a Request changes pour une virgule : ego blesse, PR cachée. Sam a dit une chose positive puis pointe le fond : correctif en vingt minutes. Variante solo : écris la revue dans `NOTES-REVIEW.md` comme à un collègue, puis applique tes remarques. L'exercice du ton reste valable.

Chez DanielCraft, on garde parfois deux formulations modèles dans le README : une pour une suggestion, une pour un vrai bloqueur. Lea les copie. Max aussi. Moins de stress, plus de clarté. Tu peux sortir de cet atelier avec tes deux modèles à toi.

Criteres et pieges

PR initiale avec description utilisable. Revue sur le fond (validation), pas seulement l'esthetique. Ton respectueux. Correctif suivi. `main` via merge. Eviter : Approve sans tester ; Request changes pour preference de virgule ; auteur qui ignore ; reviewer qui reecrit toute la PR sans discussion. Si tu te surprends a reecrire, propose plutot un pair-programming apres merge.

Erreur classique

Croire que "reviewer = trouver des fautes". Ou "auteur = defendre chaque ligne". Autre piege : discuter le style pendant une heure et rater le bug fonctionnel. Autre piege : Approve fantome pour "avancer". La confiance casse deux semaines plus tard, plus cher. Autre piege : auteur qui corrige en silence sans repondre : le reviewer ne sait pas si c'est fait.

⚠ ATTENTION

Discuter une virgule une heure et rater le bug email : mauvaise priorite. Le fond d'abord, le style ensuite.

En vrai

Copie deux formulations de commentaires que tu trouves reussies. Elles serviront de modeles. Lis-les a voix haute. Si ca sonne comme un jugement de personne, reformule sur le code. Colle-les dans le README sous "Exemples de revue".

A toi

Fais l'atelier. Livrable : lien PR (ou notes) + deux commentaires modeles + une lecon sur le ton. Range-le. Bonus : montre tes deux formulations a un collegue et demande : "tu prefere recevoir laquelle ?" Dans une semaine, applique le meme ton sur une vraie PR, meme petite.

Chapitre 16 - Atelier : conflit en equipe



Conflit : lire, choisir, tester, commit.

Les **conflits** font peur parce qu'ils arrivent souvent le vendredi, quand tout le monde veut partir. Ici, tu vas en créer un exprès, le résoudre calmement, et voir que ce n'est qu'un fichier qui demande un choix humain. Git ne "casse" pas. Il s'arrête et te demande : "deux versions, laquelle ?" Chez DanielCraft, le meilleur conflit est celui évité tout de suite. Le deuxième meilleur est celui résolu sans drama. Durée : 45 à 75 minutes. Écris le **livrable**. Sans note, le cerveau classe ça comme "lu", pas comme "su".

Lea a paniqué la première fois qu'elle a vu les marqueurs. Max aussi. Sam a appris `merge --abort` avant d'apprendre à résoudre. Abandonner proprement est une compétence. Tu vas en avoir besoin. Cet atelier ne te rend pas expert en fusion. Il te rend capable de respirer quand le terminal crie.

★ A RETENIR

Un conflit, c'est un choix de contenu, pas une panne Git. Respire, choisis, enlève les marqueurs, commit.

Deux branches touchent le même titre. Alex écrit "Atelier Git équipe". Blake écrit "Site vitrine DanielCraft". `main` reçoit Alex d'abord. Blake synchronise : conflit. Marqueurs `<<<<<<`. Tu choisis, ou tu combines. Tu enlèves les marqueurs. Tu commits. Tu respirez. La page affiche une seule version claire. Personne n'a "gagné". Le site a gagné.

Le même scénario arrive en vrai quand Lea et Max touchent `index.html` le même jour sans se le dire. Git fusionnera peut-être les zones éloignées. Sur la même ligne, il demandera un humain. Le canal ("je touche le titre aujourd'hui") reste le meilleur antidote. L'atelier, lui, entraîne le muscle quand l'antidote a raté.

But et scénario

Provoquer un conflit sur le même fichier, le résoudre, finir sur une `main` propre. Deux branches touchent le titre de `index.html` (ou le bandeau). Alex et Blake partent du même `main`. Tu peux jouer les deux rôles sur un seul compte. Idéalement, un ami joue Blake pour sentir le "vrai" frottement.

Étapes

1. Sur `main` à jour, note le titre actuel.

```
git switch main
git pull
```

1. 2. Branche Alex : `feature/titre-atelier`, modifie le titre, commit, pousse, **PR**, merge dans `main`.

1. 3. Puis branche Blake depuis `main` tire :

```
git switch main
git pull
git switch -c feature/titre-vitrine
```

Autre texte au meme endroit. Commit.

1. 4. Ramene `main` :

```
git fetch origin
git merge origin/main
```

Conflit. Marqueurs :

```
<<<<<< HEAD
titre de Blake
=====
titre d'Alex (venu de main)
>>>>>> ...
```

1. 5. Choisis ou combine ("Site vitrine DanielCraft - atelier Git"). Enleve les marqueurs. Sauve. Relis le fichier : aucun `<<<<<<` ne doit rester.

1. 6. Resolu :

```
git add index.html
git commit
```

1. 7. Pousse. PR. Description : "Resolution de conflit sur le titre, version combinee telle." Review. Merge.

1. 8. Tire `main`. Verifie le titre dans le navigateur. Sens le soulagement : c'etait ca, le monstre.

♦ ASTUCE

Si tu paniques au milieu : `git merge --abort` (ou `git rebase --abort`). Tu reviens a l'avant. Puis tu recommences a froid.

Variante rebase

Au lieu de merge : `git rebase origin/main`, resols, `git rebase --continue`, puis `git push --force-with-lease` sur la branche perso. Compare le ressenti. Le **rebase** rejoue tes commits sur une base plus recente. Le merge colle les histoires avec un noeud. Les deux demandent le meme choix de contenu. Seule la forme de l'historique change. Note ce que tu preferes. On en reparlera dans le contrat d'equipe.

Petite histoire

Lea a laisse les marqueurs dans le fichier : page cassee en prod, texte illisible avec des =====. Max a choisi au hasard sans parler : ego blesse, mauvaise version. Sam a combine et explique dans la PR : calme, trace, confiance. Regle d'equipe a ecrire : "si on touche aux memes zones, on se parle avant" + "on synchronise main souvent".

Chez DanielCraft, on chronometre parfois cet atelier. Quarante-cinq minutes suffisent si le depot est pret. Soixante-quinze si c'est la premiere fois que tu vois les marqueurs. Le but n'est pas la vitesse. C'est le calme.

Criteres et panique

Aucun <<<<<< restant. Site affiche. Historique montre la resolution. Tu expliques a voix haute ce que tu as choisi et pourquoi. Si panique : abort, pause cafe, recommence. Un conflit mal resolu pousse sur main est pire qu'un conflit non resolu encore local.

⚠ ATTENTION

Ne force-push jamais sur main pour "effacer le conflit". Tu effaces aussi le travail des autres. Resols, commit, PR.

Erreur classique

Resoudre en gardant "les deux textes empiles" sans enlever les marqueurs. Ou blamer la personne au lieu de choisir le contenu. Ou synchroniser main une fois par semaine seulement : tu transformes un petit conflit en roman. Autre piege : ouvrir la PR sans dire "j'ai resolu un conflit, voici le choix". Le reviewer doit savoir.

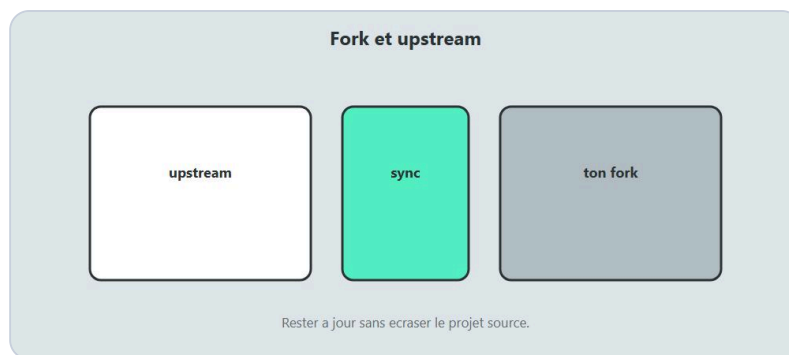
En vrai

Ecris la regle d'equipe anti-conflit en deux phrases. Colle-la dans le README. Relis-la le prochain lundi avant de toucher index.html.

A toi

Fais l'atelier. Livrable : notes du choix final + regle d'equipe + ressenti merge vs rebase si teste. Range-le. La prochaine fois que le terminal crierait, tu auras deja vecu la scene.

Chapitre 17 - Fork et upstream : contribuer ailleurs



Upstream et fork : rester à jour.

Jusqu'ici, tu étais "dans" le depot de l'équipe : tu avais le droit de pousser des branches. Sur un projet open source, ou le depot d'une autre équipe, tu n'as souvent pas ce droit. Le chemin classique : **fork**, clone, branche, PR vers le projet original. Ce n'est pas une autre religion Git. C'est le meme flux, avec une étape de copie sur ton compte. Chez DanielCraft, on contribue souvent comme ça a un outil ami : propre, petit, respectueux des regles du projet.

Tu n'as pas besoin de devenir mainteneur open source. Tu as besoin de comprendre trois boites : ton PC, ton fork, le depot original. Une fois le dessin clair, les boutons GitHub cessent d'être magiques. Ce chapitre pose le vocabulaire et le geste, sans transformer le livre en encyclopedie.

★ A RETENIR

Fork = ta copie. Upstream = l'original. Tu pousses sur ton fork, tu proposes une PR vers upstream.

Les mots

Fork : une copie du depot sur ton compte GitHub. Tu pousses sur ton fork librement. Ce n'est pas un "vol". C'est le modele prevu.

Origin : en general, apres clone de ton fork, **origin** pointe vers ton fork. C'est "chez toi" sur le serveur.

Upstream : le depot d'origine (celui de l'équipe ou du projet public). Tu le configures pour recuperer les nouveautes, pas pour y pousser directement - tu n'en as souvent pas le droit.

Pull request : tu demandes au projet original d'integrer ta branche (depuis ton fork vers upstream). Meme conversation qu'au chapitre 6. Autre cible.

Scenari concret

Lea veut corriger une typo dans la doc d'un outil qu'elle utilise. Elle ne peut pas pousser sur le depot officiel. Elle fork. Elle clone son fork. Elle cree **fix/typo-readme**. Elle corrige. Elle pousse sur son fork. Elle ouvre une PR vers le depot officiel. Le mainteneur lit, commente, merge. Lea sync son **main** depuis upstream. Fin. Une typo, un geste propre, zero drame.

Max, lui, a voulu "reformatier tout le projet" dans sa premiere PR open source. Refuse. Trop large. Sam a lu **CONTRIBUTING.md** avant. Accepte en deux jours. La difference n'était pas le talent. C'était le respect du cadre.

Les commandes d'installation

Après avoir fork sur GitHub et clone ton fork :

```
git remote -v
git remote add upstream https://github.com/ORG/PROJET.git
git fetch upstream
```

Remplace l'URL. Verifie :

```
git remote -v
```

Tu dois voir `origin` (ton fork) et `upstream` (l'original). Si tu ne vois qu'`origin`, tu n'as pas encore la fenetre sur le projet source. Ajoute upstream avant de coder longtemps : sinon tu construis sur une base qui a deja bouge.

Rester a jour avec upstream

Avant une nouvelle contribution :

```
git switch main
git fetch upstream
git merge upstream/main
git push origin main
```

(adapte le nom de branche par default si le projet utilise autre chose)

Ainsi ton `main` local et celui de ton fork suivent le projet. Ta prochaine feature part d'une base saine. Un fork qui moisit six mois, puis une PR sur une base obsolete, egal conflits inutiles et mainteneur fatigue.

♦ ASTUCE

Avant chaque contribution : `git fetch upstream` puis merge (ou rebase) de `upstream/main` dans ton `main`. Sync d'abord, code ensuite.

Le flux de contribution

```
git switch main
git pull origin main
git switch -c fix/typo-readme
# ... corrections, commits ...
git push -u origin fix/typo-readme
```

Puis sur GitHub : Pull request de `ton-compte:fix/typo-readme` vers `ORG/PROJET:main`. Lis les commentaires des mainteneurs. Corrige. Sois patient. Une contribution refusee n'est pas une attaque personnelle : parfois le timing, parfois la direction du projet. Tu pousses sur ta branche de ton fork ; la PR se met a jour. Tu n'as pas besoin d'accès write sur upstream pour iterer.

Quand c'est merge, tu sync ton `main` depuis upstream (fetch + merge) pour rester aligne, et tu peux supprimer ta branche de feature sur ton fork. Meme hygiene qu'en equipe interne.

Bonnes manieres

PR petite. Sujet unique. Message et description clairs. Respecter le style du projet. Ne pas reformater deux cents fichiers "en passant". Ne pas ouvrir dix PR de bruit le premier jour. Si le projet a un fichier `CONTRIBUTING.md`, lis-le. Ca evite quatre-vingts pour cent des malentendus. Licence, code de conduite, templates de PR : ce n'est pas de la paperasse. C'est le mode d'emploi pour etre entendu.

⚠ ATTENTION

Une premiere PR qui reformatte tout le depot "en passant" est souvent refusee. Un sujet, une intention, un diff lisible.

Et dans une entreprise ?

Parfois tu n'as pas de fork : tu as directement acces au depot, et le chapitre 2 suffit. Le fork sert surtout quand les droits d'ecriture sont limites, ou pour isoler des experiences. Comprendre fork/upstream reste utile des que tu touches l'open source - et parfois en interne, si l'org te met en lecture seule plus process PR.

Meme avec acces write, certains forkent pour tenter une idee folle sans polluer les branches de l'equipe. Moins courant en petite boite, courant en open source. L'idee reste : isolation plus proposition.

Dans une org GitHub, on t'ajoute parfois en write sur le depot : pas besoin de fork. On t'ajoute parfois seulement en lecture plus process fork/PR : besoin de fork. Demande quel modele est en place. Ne suppose pas. Lea a perdu une matinee a chercher un bouton "push" qui n'existait pas. Sam lui a dit : "tu es en lecture. Fork."

Petite histoire

Lea a fork un outil, oublie `upstream`, code trois semaines, ouvert une PR. Le projet avait avance de quarante commits. Conflits partout. Elle a sync, recommence, PR minuscule. Accepte. Max a pousse vers `upstream` par erreur de remote : refuse par le serveur, ego un peu blesse, lecon nette. Chez DanielCraft, on dessine les trois boites au tableau avant la premiere contribution externe. Dix minutes. Beaucoup de clarte.

Erreur classique

Pousser vers `upstream` alors que tu n'as pas les droits (erreur) ou, pire, croire que `git push` sans remote precise ira "au bon endroit". Verifie avec `git remote -v`. Autre piege : laisser ton fork moisir six mois. Ou ouvrir une PR geante le premier jour pour "impressionner". Tu impressionnes surtout le bouton Close.

En vrai

Fork un petit projet (meme un depot demo), ajoute `upstream`, synchronise `main`, ouvre une PR minuscule sur ton propre fork vers un second depot de test si besoin, pour voir les boutons GitHub. L'important est de voir origin vs upstream une fois dans l'interface. Le corps retient mieux que le schema seul.

A toi

Dessine sur papier trois boites : ton PC, ton fork, le depot original. Fleches : fetch upstream, push origin, PR vers upstream. Si le dessin est clair, le modele est clair. Garde le papier. Tu le ressortiras la prochaine fois que quelqu'un dira "fork-moi ca".

Chapitre 18 - Hygiene des secrets : jamais de cles dans Git



.env hors git, cles hors historique.

Un commit peut vivre longtemps. Il se copie sur les machines, les forks, les backups, les CI logs. Si tu commits une **cle** API, un mot de passe, un fichier `.env` rempli, tu as publie un **secret**. Meme si tu "effaces" le fichier au commit suivant, l'histoire le contient encore. Les robots scannent les depots publics en permanence. Les depots prives fuient aussi : fork demain, stagiaire apres-demain, capture d'ecran un jour. Chez DanielCraft, on prefere un projet un peu moins "pratique a cloner" qu'un projet qui fuit des cles.

Ce chapitre est non negociable. Pas un "nice to have". Pas un chapitre "si tu as le temps". Si tu ne retiens qu'une phrase du livre apres le flux : les secrets ne vivent pas dans Git.

★ A RETENIR

Jamais de secrets dans Git. Invalide d'abord, nettoie ensuite. Un `git rm` seul ne suffit pas.

Ce qu'on ne commit pas

Fichiers `.env`, `.env.local`, cles privees SSH, fichiers `credentials.json`, dumps de base avec donnees reelles, exports de mots de passe, tokens de bot, certificats prives. Les exemples de config, oui. Les vraies valeurs, non. Un screenshot de dashboard cloud avec une cle visible dans une PR, non. Un `console.log` qui affiche le token, non. Un fichier de fixture "pour que ca marche chez toi" avec la vraie cle, non.

Lea a un jour commit un dump "anonyme" qui contenait encore des emails clients. Max a commit une cle "de dev" qui ouvrirait trop. Sam a commit un `.pem` "temporairement". Trois scenes, une meme lecon : temporaire dans Git, c'est permanent dans l'histoire.

.gitignore est ton ami

Dans le depot :

```
.env
.env.*
!.env.example
```

Tu commits `.env.example` avec des fausses valeurs :

```
API_KEY=remplace_moi
DATABASE_URL=postgres://user:pass@localhost:5432/app
```

Chaque developpeur copie vers `.env` en local et remplit. Le `.env` reste hors Git. `.env.example` est un contrat : il liste les variables necessaires sans valeurs sensibles. Un nouveau developpeur (ou toi sur une nouvelle machine) sait quoi remplir. Si l'exemple ment (variable manquante), l'onboarding souffre et quelqu'un recommit un vrai `.env` "pour aider". Garde l'exemple juste.

♦ ASTUCE

Commit `.env.example` avec des fausses valeurs. Ignore `.env`. Au premier jour d'un collegue, il copie, remplit, avance - sans jamais pousser de vrai secret.

Comment l'equipe partage les secrets

Pas via un commit "temporaire". Via un gestionnaire (coffre, secrets GitHub, variable d'environnement du serveur, outil d'equipe). Le canal "je te colle la cle sur Discord" est deja risque : prefere un outil qui expire, qui audit, qui restreint. La **CI** lit les secrets depuis les reglages du depot, pas depuis le code. Le fichier yaml decrit le job. Les valeurs sensibles vivent ailleurs.

En review, un oeil sur les fichiers suspects : `.pem`, `.env`, `id_rsa`, longs tokens dans le source. Un reviewer qui attrape ca avant le merge sauve des soirees. Les branches protegees et la CI n'empechent pas a elles seules un secret dans une PR. L'humain et `.gitignore` comptent. Approve fantome egal secret qui passe.

Si le mal est fait

1. Revoque ou regenere la cle tout de suite chez le fournisseur (cloud, API, DB).
2. Enleve le secret du code actuel.
3. Consideres l'historique : selon la gravite, filtre l'historique (outils specialises) ou traite le depot comme compromis pour cette cle.
4. Previens l'equipe. Transparence bat honte silencieuse.

La priorite absolue : invalider le secret. Un `git rm` seul ne suffit pas. "Je vais l'enlever au prochain commit" laisse le tresor dans le passe. Les scanners le trouvent. Les attaquants aussi.

⚠ ATTENTION

"C'est un depot prive, donc OK" est faux. Prive aujourd'hui, fork demain, fuite un jour. Revoque des qu'un secret a circule.

Exemple Max

Max ajoute l'envoi d'email. Le fournisseur donne une cle. Max met la cle dans `config.js` "pour tester". Il commit. Il pousse. La cle est publique sur le remote. Scenario cauchemar classique. La bonne version : `process.env.MAIL_API_KEY` (ou equivalent) plus `.env` ignore plus exemple dans `.env.example`. Cinq minutes de plus. Des semaines de moins a gerer un incident.

Lea, en review, a vu un long token dans un fichier JS. Elle a demande changes. Max a revoke, corrige, remercie. Ce n'etait pas un tribunal. C'etait un filet. Chez DanielCraft, on applaudit le reviewer qui attrape un secret comme on applaudit celui qui trouve un bug de paiement.

Checklist avant le premier push d'une feature API

Est-ce que la cle est dans le code source ? Dans un screenshot de la PR ? Dans un log `console.log` ? Dans un fichier de fixture committe ? Si oui a l'une de ces questions, stop. Sors le secret. Regenere si elle a deja circule. Relis le diff une derniere fois avec les yeux "secret hunter". Cinq minutes ici epargnent une nuit blanche.

Petite histoire

Sam a commit une cle "temporairement" un mardi. Mercredi, un bot a ouvert une issue "exposed credential". Jeudi, rotation de toutes les cles proches, audit, excuses. Vendredi, `.gitignore` renforce et phrase dans le README. L'equipe a gagne une cicatrice utile. Mieux vaut lire ce chapitre avant la cicatrice.

Erreur classique

"C'est un depot prive, donc OK." Non. "Je vais l'enlever au prochain commit." L'histoire garde. "C'est juste une cle de dev." Souvent la cle de dev ouvre encore trop. Autre piege : coller la cle dans le titre de la PR ou dans un commentaire GitHub "pour que Max teste". Le commentaire reste. Le titre reste. Traite GitHub comme un lieu public avec memoire.

En vrai

Audit rapide :

```
git ls-files | findstr /i "env pem key credential"
```

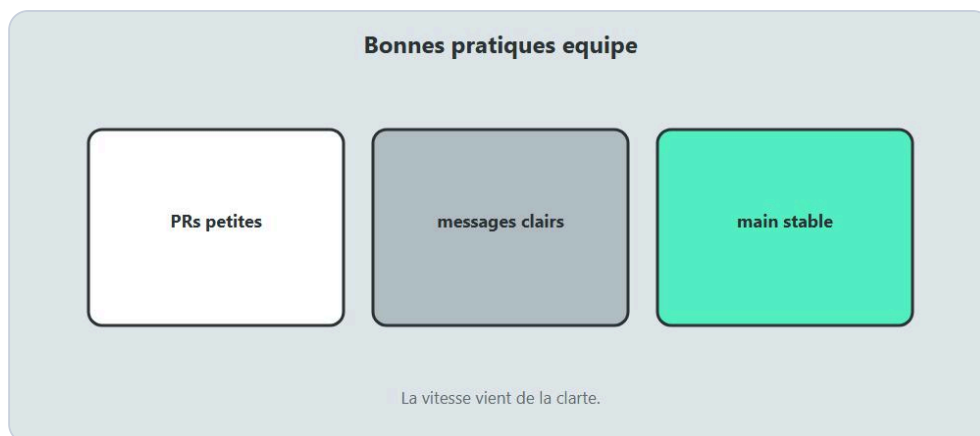
(sous Linux/mac : `git ls-files | grep -iE 'env|pem|credential'`)

Verifie `.gitignore`. Tourne les cles qui ont fuit ne serait-ce qu'une fois en local douteux. Regarde aussi les anciennes PR et les screenshots. L'hygiene depasse Git : c'est une posture.

A toi

Ajoute (ou verifie) `.env.example` plus regles `.gitignore` sur ton depot de test. Ecris dans le README : "Jamais de secrets dans Git. Les cles vivent hors depot." Lis cette phrase a chaque nouvelle integration d'API. Puis respire. Tu viens de fermer une porte que beaucoup laissent ouverte.

Chapitre 19 - Bonnes pratiques d'equipe



PRs petites, messages clairs, main stable.

Les outils ne sauvent pas une equipe qui ne se parle pas. Les pratiques ci-dessous sont le minimum de politesse technique pour deux a cinq personnes. Adapte. Ecris. Rappel. Chez DanielCraft, on repete souvent : clarte, filets, bienveillance. Git est un moyen. Le produit et les humains sont la fin. Si une pratique Git rend tout le monde miserable sans gain de fiabilite, changez la pratique.

Tu n'as pas besoin de quinze regles. Tu as besoin de trois a cinq **pratiques** tenues. Les tenir bat les collectionner. Ce chapitre est une checklist vivante, pas un roman ISO. Lis-le une fois. Choisis. Ecris dans le README. Relis en retro.

★ A RETENIR

Quelques regles tenues battent vingt regles ignorees. Ecris le contrat leger. Tiens-le.

Ecrire le contrat leger

Un README section "Comment on travaille" avec : branches, **PR** obligatoires, qui review, rebase ou merge, comment on release, ou vivent les secrets. Une demi-page. Pas un roman. Si ce n'est pas ecrit, chacun invente. Puis s'etonne. Lea invente "on pousse sur main si c'est petit". Max invente "rebase partout". Sam invente "pas besoin de review". Trois inventions, un vendredi triste.

Le contrat leger n'est pas fige. Apres deux semaines, ajustez une phrase. Une seule amelioration par retro suffit. Le document qui change un peu reste vivant. Celui qui ne change jamais devient decoratif.

Petites PR, souvent

Livre des tranches. Review plus facile. Rollback plus simple. Moral plus haut. La "grosse PR du mois" est un anti-pattern courant. Cent vingt lignes se lisent. Deux mille cinq cents se feignent. Decoupe meme si ca demande deux merges. Le calendrier te remerciera. Le reviewer aussi.

Synchroniser tot

Tire **main** souvent. Parle si tu touches une zone chaude. Evite les conflits-surprise. Le silence n'est pas de la concentration heroique : parfois c'est de la collision en preparation. Un message dans le canal coute dix secondes. Un conflit surprise sur le meme fichier coute une heure. Tu le sais depuis le chapitre 2. Tiens-le.

Protéger ce qui compte

`main` `protégée`. Secrets hors Git. `CI` légère verte avant merge. Tags pour les versions livrées. Ces quatre gestes évitent une catégorie entière de vendredis tristes. Tu peux les activer en une après-midi sur un dépôt de test. Habitude avant vitesse. Filet avant héroïsme.

♦ ASTUCE

Quatre filets minimum : `main` protégée, secrets hors Git, CI légère, tags de release. Active-les tout, pas "quand on aura le temps".

Review comme un collègue, pas comme un juge

Aide. Questionne. Propose. Remarque le bien. Réponds vite. L'ego hors du diff. Approuve conscient ou pas. Approuve. Request changes pour les vrais bloqueurs, pas pour une virgule. L'auteur écrit "comment tester". Le reviewer suit "comment tester". Deux gestes, une équipe qui respire.

Noms clairs partout

Branches, commits, PR, tags. Le futur lecteur est un humain fatigué à 18h. Écris pour lui. `feature/page-tarifs` bat `max-wip`. "Corrige le timeout session" bat "fix". `v1.2.0` bat "version de mardi". La clarté est une forme de gentillesse.

Une intention à la fois

Un bug urgent ne voyage pas caché dans une refonte. Deux branches. Deux PR. Le calendrier te remerciera. Cherry-pick si vraiment une cerise doit partir avant le panier. Bisect si tu cherches le commit casse. Outils de précision, pas de panique.

Sabbat du force-push

Force-push seulement sur ta branche perso, avec `--force-with-lease`, en conscience. Jamais sur `main`. Jamais "pour voir" sur la branche d'un autre. Annonce dans le canal si la branche était déjà partagée. Le silence plus force-push égal egos et histoires écrasées.

⚠ ATTENTION

`--force` sur `main` ou sur la branche d'un collègue sans annonce : non. Préfère `--force-with-lease` sur ta branche perso seulement.

Documenter les incidents sans humiliation

Quand ça casse : chronologie, cause, correctif, prévention. Pas de pile au milieu de la pièce. Une équipe qui apprend plus vite gagne plus qu'une équipe qui punit. Max a cassé la prod un vendredi. L'équipe a écrit "protection main des le jour 1". Pas "Max est nul". Le process a gagné. Max aussi.

Rituels hebdomadaires legers

Dix minutes lundi : branches mortes a supprimer ? CI rouge ignoree ? Secrets tourne apres un depart ? Une seule question par semaine suffit a eviter la poussiere. L'onboarding d'un nouveau : README "Comment on travaille", un acces, une petite PR de doc ou de typo pour vivre le flux. La premiere PR est un rituel d'equipe, pas un examen.

Quand casser une regle : rarement, explicitement, annonce dans le canal, remets la regle apres. Une regle sans exception jamais discutee devient du theatre. Une exception tous les jours devient l'absence de regle.

Petite histoire

Lea a copie le process d'une grosse boite : trois approvals, douze checks, signatures. L'equipe de trois s'est bloquee. Ils ont simplifie. Flux tenu. Reviews utiles. `main` protegee. CI d'un job. Trois pratiques non negociables dans le README. Deux mois plus tard, toujours tenues. Chez DanielCraft, on dit : le juste milieu, c'est quelques regles tenues, pas vingt ignorees.

Erreur classique

Copier le process d'une FAANG a trois personnes. Ou n'avoir aucun process. Ou empiler bots et hooks avant d'avoir un flux tenu. Autre piege : ecrire quinze non negociables et n'en tenir aucune. Trois, tenues, battent quinze, ignorees.

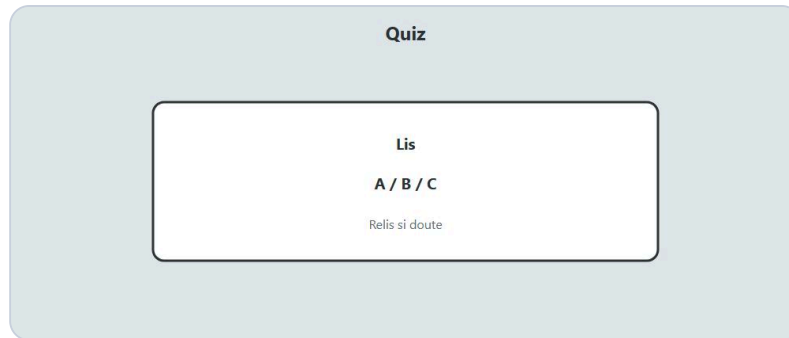
En vrai

Fais une retro de vingt minutes apres deux semaines de mini-projet. Qu'est-ce qui a frotte ? Quelle phrase ajouter au README ? Une seule amelioration suffit par retro. Mesure simple sans dashboard : push directs sur `main` (doit tendre vers zero), temps avant premier regard sur une PR, secrets revoques apres incident (doit etre "tous", tout de suite). Un ressenti honnete en retro suffit au debut.

A toi

Choisis trois pratiques de ce chapitre et ecris-les comme "non negociables" pour ton equipe. Trois, pas quinze. Les tenir bat les collectionner. Colle-les sous l'ecran. Relis-les avant la prochaine PR.

Chapitre 20 - Quiz final



Vérifier sans inventer.

Pas de piège. Relis une fois si besoin. Le but : vérifier que tu peux avancer en équipe sans paniquer. Chez DanielCraft, le vrai test, c'est la prochaine **PR** calme, pas le 12/12. Douze questions, les thèmes du livre : **flux**, branches, rebase, historique, revue, protection, CI, tags, cherry-pick, bisect, **secrets**. Si tu bloques, note le numéro et relis le chapitre. Pas de honte. Un miroir.

Lea note ses erreurs et rouvre deux chapitres. Max refait l'atelier flux. Sam utilise ce QCM en fin de module. Toi, tu coches sans tricher. Une première passe honnête vaut mieux qu'un score maquillé. Le but, c'est la carte des chapitres à rouvrir. Fais le quiz, note ton score, rejoue dans une semaine. Le vrai progrès se voit à la deuxième passe.

★ A RETENIR

Le quiz est un miroir, pas un jugement. Note le frein, relis le chapitre, agis quinze minutes.

Questions

1. 1. Dans un flux feature vers main, que fait-on en général juste avant de créer une branche ?

- A) Supprimer le dépôt distant
- B) Se placer sur `main` et faire `git pull`
- C) Faire `git push --force` sur `main`

1. 2. Une bonne raison de garder des branches feature courtes :

- A) Pour colorier GitHub
- B) Pour limiter les divergences avec `main` et les gros conflits
- C) Parce que Git interdit les branches longues

1. 3. Le rebase, en une idée simple, c'est surtout :

- A) Effacer GitHub
- B) Rejouer tes commits sur une autre base pour un historique plus linéaire
- C) Remplacer `git status`

1. 4. Sur une branche `main` partagée, que faut-il presque toujours éviter ?

- A) Les pull requests

- B) Les tags annotes
- C) Un rebase suivi d'un force push

1. 5. Un message de commit utile raconte surtout :

- A) La liste des fichiers sans intention
- B) Le pourquoi (l'intention) du changement
- C) Le mot de passe de la base

1. 6. Dans une description de PR utile, on trouve souvent :

- A) Uniquement "fix"
- B) But, changements, et comment tester
- C) La cle API en clair

1. 7. Protéger `main` sert surtout à :

- A) Interdire Git localement
- B) Imposer un passage par PR (et souvent review / CI) avant d'intégrer
- C) Ralentir Internet

1. 8. Une CI légère sur les PR, c'est surtout pour :

- A) Remplacer toute review humaine
- B) Lancer automatiquement des vérifs (tests, lint, build) et signaler rouge/vert
- C) Générer des logos

1. 9. Un tag `v1.2.0` sert surtout à :

- A) Remplacer les branches
- B) Nommer de façon stable un commit (souvent une version livrée)
- C) Effacer l'historique

1. 10. `git cherry-pick` sert surtout à :

- A) Copier un commit précis sur une autre branche
- B) Supprimer GitHub
- C) Remplacer `.gitignore`

1. 11. `git bisect` sert surtout à :

- A) Punir un collègue en public
- B) Trouver par recherches successives le commit qui a introduit un bug
- C) Merger automatiquement

1. 12. La bonne hygiène pour une cle API :

- A) La commit dans `config.js` "temporairement"
- B) La mettre hors Git (ex: `.env` ignore) et fournir un `.env.example`
- C) L'écrire dans le titre de la PR

Reponses

1-B, 2-B, 3-B, 4-C, 5-B, 6-B, 7-B, 8-B, 9-B, 10-A, 11-B, 12-B.

Si tu as rate la 1 ou la 2 : chapitre 2 et 3. Rate la 3 ou la 4 : chapitre 4. Rate la 5 : chapitre 5. Rate la 6 : chapitre 6 et atelier revue. Rate la 7 : chapitre 7. Rate la 8 : chapitre 8. Rate la 9 : chapitre 9. Rate la 10 : chapitre 10. Rate la 11 : chapitre 11. Rate la 12 : chapitre 18, sans detour. Une erreur pointe un chapitre, pas une faute de caractere.

Score

- 11-12 : pret pour de vraies collabos. Tiens le contrat leger cette semaine.
- 9-10 : tres bien, relis le chapitre qui a coince, puis une micro-PR de test.
- 7-8 : relis flux, revue, protection, secrets. Refais l'atelier flux si besoin.
- 5-6 : refais ateliers flux et conflit. Pas de honte : les mains ancrent.
- moins de 5 : reprends chapitres 1 a 7 sans stress. Le quiz attendra.

Petite histoire

Lea a eu 8/12. Elle a relu le rebase, puis tenu une PR propre. Max a rate les secrets : il a verifie son `.gitignore` le soir meme. Sam a eu 11/12 et a quand meme protege `main` sur son depot de test. Le quiz est un miroir. Chez DanielCraft, on aime ces miroirs sans humiliation.

Lea a aussi note : "j'ai coche trop vite sur cherry-pick". Elle a relance `git show` sur un depot de jouet. Cinq minutes. Le geste est revenu. Max a rate la CI : il a ecrit la phrase README avant meme d'avoir un workflow. L'intention d'abord. Sam a montre le quiz a un stagiaire : douze questions, zero piege, une carte claire.

Erreur classique

Tricher pour "avoir 12". Ou lire les reponses avant. Ou noter le score sans action. Autre piege : croire que 12/12 remplace une PR calme la semaine suivante. Le quiz mesure le souvenir. La collabo mesure le geste.

⚠ ATTENTION

Un 12/12 sans depot de test et sans PR recente, c'est un score papier. Ancre par un geste de quinze minutes.

En vrai

Note ton score et le chapitre a relire. Fais une action de quinze minutes liee (ouvrir une PR de test, ajouter `.env.example`, ecrire la regle d'equipe, proteger `main`). Le geste ancre mieux qu'une relecture passive. Si tu as 12/12, choisis quand meme une pratique a renforcer : personne n'est "fini".

A toi

Ecris : score / frein principal / prochaine pratique a tenir cette semaine. Colle sous l'ecran. Dans sept jours, rejoue seulement les questions ratees. Puis coche : une PR avec description, ou `main` protegee, ou secrets verifies. Le quiz devient un plan, pas un trophée.

Chapitre 21 - Bravo



Bravo. Tu sais travailler Git en equipe.

Tu as fini Git en equipe. Pas "tu connais toutes les options de toutes les commandes". Mieux : tu sais collaborer sans te marcher dessus. Chez DanielCraft, on mesure le progres a une chose simple : est-ce que ton prochain projet a plusieurs sera plus calme que le precedent ? Si oui, le livre a servi. Si non, relis le flux et la revue. Le reste s'accroche dessus.

Tu as une carte. **Flux**. Branches simples. Rebase et merge sans dogme. Historique lisible. Revue humaine. **main** **protegee**. CI legere. Tags et releases. Cherry-pick pour une cerise urgente. Bisect pour trouver le commit casse. Un mini-projet. Des ateliers. Fork et upstream. **Secrets** hors du depot. Des pratiques d'equipe tenables. Ce n'est pas un diplome. C'est un kit de survie pour les mardis et les vendredis.

★ A RETENIR

Connaitre Git, c'est le moteur. Savoir collaborer, c'est la route. Tu as les deux. Tiens le contrat leger.

Lea pousse sa branche. Max demande un regard. Sam dit merci au reviewer. Quelqu'un tire **main**. On recommence. Ce rythme, c'est le livre en une scene. Pas un examen. Une **habitude**. Un vendredi soir sans panique vaut mieux qu'un score parfait au quiz. Tu n'as pas besoin d'etre heroique. Tu as besoin d'etre regulier : petites PR, reviews utiles, filets techniques, secrets dehors.

Imagine le tableau blanc du chapitre 1. **main** distant, c'est le tableau officiel. Lea, Max et Sam ont chacun une copie. Sans flux, chacun dessine et s'ecrase. Avec flux : tire, dessine sur ton coin, propose, regarde, colle, nettoie. Tu connais maintenant chaque geste. Il reste a le vivre demain matin.

Petite histoire

Lea avait peur des conflits. L'atelier les a demystifies. Max poussait sur **main** "parce que c'est plus vite". Une protection de branche l'a sauve d'un vendredi noir. Sam a commit une cle "temporairement". Le chapitre secrets l'a rattrape avant la prod. Trois peurs, trois filets. Si tu bloques, reviens au flux (ch. 2) et a la revue (ch. 6). Presque tous les frottements viennent de la.

Chez DanielCraft, on a vu des equipes talentueuses souffrir faute de contrat leger. Et des equipes moyennes livrer calmement avec cinq phrases dans un README. Le talent compte. Le rythme compte plus souvent.

♦ ASTUCE

Cette semaine, une seule pratique : protege `main` , ou ouvre une PR avec description, ou verifie `.gitignore` . Une. Tiens-la. Puis ajoute la suivante.

Ce que tu peux faire maintenant

Tenir un README de collaboration. Ouvrir des PR petites et claires. Reviewer sans blesser. Refuser le push direct sur `main` . Taguer une version. Retrouver un bug dans l'histoire. Contribuer via fork. Garder les clefs dehors. Applique une pratique a la fois. Garde le contrat léger a jour. Quand ça frotte, ajustez sans ego. La suite (hooks avances, monorepo, changelog auto) s'accrochera sur ce socle. Sans socle, les outils avances empilent du bruit.

Tu peux aussi transmettre. Montre le flux a un stagiaire. Fais-lui vivre l'atelier conflit une fois. Lis avec lui la phrase sur les secrets. Enseigner ancre. L'équipe gagne deux fois. Si tu es seul, écris un court message a un ami : "voilà comment on travaille chez nous". Formuler force la clarté.

Checklist poche a coller sous l'écran : tire `main` le matin ; branche nommée ; commits qui disent le pourquoi ; PR avec comment tester ; review utile ; merge ; tire `main` ; nettoie la branche ; tague quand tu livres ; secrets hors Git. Dix lignes. Une vie d'équipe.

Erreur classique

Fermer le livre et revenir aux anciennes habitudes "juste pour ce hotfix". Ou empiler bots et hooks avant d'avoir un flux tenu. Autre piège : attendre d'être expert DevOps pour protéger `main` . Tu n'as pas besoin d'être expert. Tu as besoin d'un filet simple aujourd'hui. Autre piège : croire que le quiz 12/12 te dispense de la prochaine review bienveillante.

⚠ ATTENTION

Le "juste pour ce hotfix" sur `main` sans PR, c'est souvent le début de la régression. Passe par une branche, même a 19h.

En vrai

Cette semaine : protege `main` sur un depot (même de test), ouvre une PR avec description, demande un regard. Une seule pratique. Tiens-la. Note ce qui change dans ton calme. Le calme est le vrai indicateur. Dans quinze jours, ajoute une deuxième pratique (CI légère, ou `.env.example` , ou tags). Le rythme se construit par couches.

Mission

Écris ton contrat d'équipe en cinq lignes. Colle-le dans le README. Fais une PR qui le respecte. Puis respire. Tu n'as pas "fini Git". Tu as commencé a collaborer proprement. C'est plus rare. C'est plus utile.

Cinq lignes type : "On tire main avant chaque branche. On travaille en feature/fix courtes. Toute integration passe par une PR avec comment tester. main est protégée, une review minimum. Jamais de secrets dans Git ; on tague les livraisons utiles." Adapte. Signe. Tiens.

A toi

Coche : flux tenu / PR avec description / `main` protégée ou plan pour / secrets vérifiés. Si une case manque, fais-la avant le prochain livre. Pousse ta branche. Demande un regard. Merci ton reviewer. Tire `main`. Recommence. Bravo. Tu sais travailler à plusieurs avec Git.

Petite scene finale

Lea ouvre une PR courte le mardi. Max laisse un commentaire utile. Sam merge après vert CI. Personne ne crie. Chez DanielCraft, ce silence calme, c'est le vrai bravo. Pas un score. Un rythme.

Tu as les briques. Maintenant, construis.